

Wolverine SIH

*Cross-Platform Entity Resolution and
Cryptographic Trust Infrastructure
for Synthetic Intelligence Analysis*

Technical Report and Research Monograph

Smart India Hackathon 2025

Prototype Documentation — Evidence-Classified

*This document employs a five-tier evidence classification framework
(Class A through Class E) to explicitly distinguish verified implementation
from simulated boundaries and missing experiments.*

August 20, 2026

Abstract

The Wolverine ecosystem, developed for the Smart India Hackathon (SIH), is a deterministic prototype platform designed to demonstrate cross-platform intelligence gathering and entity resolution. Addressing the inherent isolation of distinct moderation regimes on heterogeneous web platforms, the project implements a simulated ecosystem of five distinct community networks (Atlas Market, Briar Bazaar, Cinder Exchange, Drift Forum, and Ember Commons) initialized with a synthetically generated dataset of 50,000 canonical personas.

This technical report documents the architectural, data, and analytical models implemented in the Wolverine prototype. Specifically, it details the pipeline for extracting, normalizing, and resolving identities across these disparate platforms using heuristic entity resolution—including Jaro-Winkler alias similarity and weighted temporal/behavioral cues. To mitigate data overload and assist human analysts, the system integrates a deterministic Retrieval-Augmented Generation (RAG) subsystem with strict prompt sanitization rules.

Crucially, this monograph explicitly delineates the boundaries of the prototype. While the system architecture demonstrates advanced concepts—such as multi-hop graph traversal and cryptographic trust layers—key elements like the graph projection and evaluation metrics are implemented as in-memory simulations or mocked outputs designed strictly for controlled demonstration. By strictly separating verified implementation from conceptual specifications, this report provides a rigorously honest academic evaluation of the Wolverine SIH submission.

Acknowledgements

We express our gratitude to the Smart India Hackathon (SIH) organizers, mentors, and the open-source software engineering community whose foundational tools and libraries enabled the construction of this research prototype.

Special acknowledgement is extended to the developers and maintainers of the open-source technologies utilized throughout this platform, including PostgreSQL, MySQL, Redis, MinIO, Docker, Tor, Node.js, Django, Laravel, Chi, Axum, and Ollama.

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
BFS	Breadth-First Search
BFT	Byzantine Fault Tolerance
CDC	Change Data Capture
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
CTE	Common Table Expression
DDoS	Distributed Denial of Service
DMZ	Demilitarized Zone
DOM	Document Object Model
ER	Entity Resolution
EVM	Ethereum Virtual Machine
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GLSL	OpenGL Shading Language
GNN	Graph Neural Network
gRPC	gRPC Remote Procedure Call
HITL	Human-in-the-Loop
HSM	Hardware Security Module
JSON	JavaScript Object Notation
KMS	Key Management Service
LLM	Large Language Model
NLP	Natural Language Processing
OOM	Out of Memory
OSINT	Open Source Intelligence
PBFT	Practical Byzantine Fault Tolerance
PII	Personally Identifiable Information
PR	Precision-Recall
PRNG	Pseudorandom Number Generator
R3F	React Three Fiber
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFC	Request for Comments
ROC-AUC	Receiver Operating Characteristic — Area Under the Curve
SHA	Secure Hash Algorithm
SIH	Smart India Hackathon
SQL	Structured Query Language
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TCP/IP	Transmission Control Protocol / Internet Protocol
TOCTOU	Time-of-Check to Time-of-Use
TP	True Positive
ULID	Universally Unique Lexicographically Sortable Identifier
UUID	Universally Unique Identifier
WDB	Wolverine Database (wolverine-db)

Contents

Acknowledgements	iii
List of Abbreviations	vi
1 Introduction	1
1.1 Problem Context	1
1.2 Problem Formulation	1
1.3 Motivation	2
1.4 Objectives	2
1.5 Contributions	3
1.5.1 Architectural Contribution	3
1.5.2 WolverineDB Cryptographic Trust Layer	3
1.5.3 Dual-Frontend Architecture	3
1.5.4 Evidence Classification Framework	3
1.5.5 Engineering Contribution	4
1.5.6 Analytical and Methodological Contribution	4
1.6 Scope and Boundaries	4
1.7 Report Organization	4
2 Domain and Technical Foundations	7
2.1 Heterogeneous Intelligence Records	7
2.2 Canonical Representation	7
2.3 Record Linkage and Entity Resolution	8
2.4 String Similarity	8
2.5 Temporal Proximity	9
2.6 Graph Representation	9
2.7 Retrieval-Augmented Generation and Safety	9
2.8 Cryptographic Integrity and Trust	10
2.9 Synthetic Data and Ground Truth	10
2.10 Chapter Synthesis	10
3 Related Work and Research Context	11
3.1 Record Linkage and Entity Resolution	11
3.2 Cross-Platform Identity Resolution	11
3.3 String Similarity Metrics	12
3.4 Graph-Based Intelligence Analysis	12
3.5 Cyber Threat Intelligence	12
3.6 Synthetic Data and Ground Truth	13
3.7 Retrieval-Augmented Generation and Prompt Security	13
3.8 Cryptographic Integrity and Distributed Trust	13
3.9 Container Security and Microservices	14

3.10	Threat Modeling	14
3.11	Tor and Onion Architecture	14
3.12	Research vs Engineering Gap	14
3.13	Comparative Synthesis	15
4	System Requirements and Threat Model	17
4.1	Stakeholders and Actors	17
4.2	System Assets	17
4.3	Functional Requirements	17
4.4	Non-Functional Requirements	18
4.5	Trust Boundaries	19
4.6	Threat Model	19
4.7	Failure and Abuse Cases	19
4.8	Ethical and Operational Constraints	19
5	System Architecture	21
5.1	Architectural Objectives	21
5.2	System Context	21
5.3	Environment Topology and Trust Zones	22
5.3.1	wolverine-dmz	23
5.3.2	wolverine-internal	23
5.3.3	wolverine-truth	23
5.4	Container-Level Architecture	23
5.5	Service Dependency Graph	23
5.6	Data-Flow Architecture	25
5.7	Trust Boundaries	25
5.8	Failure Architecture	25
5.9	Architectural Trade-offs	26
5.9.1	In-Memory Graph Projection	26
5.9.2	WolverineDB Simulation Boundaries	26
6	Data Architecture and Database System	27
6.1	Data Model Overview	27
6.1.1	Prisma Schema Entities	27
6.2	Heterogeneous Source Schemas	28
6.3	Canonicalization Pipeline	28
6.4	Provenance and Integrity	29
6.5	Database and Storage Topology	29
6.6	Data Consistency and the Outbox Pattern	29
6.7	Data Lifecycle	30
6.8	WolverineDB Integration	30
6.9	Database Failure Modes	31
6.10	Data Architecture Trade-offs	31
7	Intelligence and Analytical Methodology	33
7.1	Analytical Pipeline Overview	33
7.2	Blocking	33
7.3	Jaro Similarity	33
7.3.1	Mathematical Formulation	33
7.4	Jaro-Winkler Similarity	34
7.4.1	Mathematical Formulation	34
7.5	Temporal Proximity	34

7.5.1	Mathematical Formulation	34
7.6	Activity Overlap (Heuristic Placeholder)	34
7.7	Cross-Site Cue	34
7.8	Composite Entity Score	35
7.8.1	Worked Example: Complete Composite Score	35
7.9	Decision Logic	35
7.10	Graph Projection	36
7.11	Breadth-First Search (BFS)	36
7.11.1	Mathematical Formulation	36
7.11.2	Worked Example: BFS Graph Traversal	36
7.12	Code-to-Math Mapping	36
7.13	RAG Pipeline	37
7.14	Prompt-Injection Mitigation	37
7.14.1	Worked Example: Sanitization	37
7.14.2	Residual Risk	37
7.15	Deterministic Fallback	37
7.16	Algorithmic Limitations	37
8	Implementation	39
8.1	Repository Architecture	39
8.2	Backend Modules (<code>wolverine-sih</code>)	39
8.3	Role-Based Access Control	39
8.4	API Routes Configuration	40
8.5	Pipeline Processor	40
8.6	Entity Resolver Implementation	41
8.6.1	Blocking Strategy	41
8.6.2	Decision Logic	41
8.7	Graph Projector Implementation	41
8.8	RAG Implementation	42
8.9	WolverineDB Implementation	42
8.10	Frontend Implementation	42
8.11	Infrastructure and Configuration	42
8.12	Testing	42
8.13	Failure Handling	43
8.14	Implementation Trade-offs	43
9	WolverineDB: Cryptographic Trust Layer	45
9.1	Overview and Design Philosophy	45
9.2	Specification Taxonomy	45
9.3	Cryptographic Primitives	46
9.4	Sentinel Security Module	46
9.5	Trust Network and BFT Consensus	46
9.6	Database Adapters	47
9.7	KMS and Signing Providers	47
9.8	EVM Anchoring	47
9.9	BFT Hardening (v1.3.0)	47
9.10	Replay Protection	48
9.11	Security Vulnerabilities and Limitations	48
9.12	Test Architecture	49

10 Frontend Systems Architecture	51
10.1 Frontend Architecture Overview	51
10.2 Vzeya: Cinematic Presentation Layer	51
10.2.1 Technology Stack	51
10.2.2 Routing and Component Hierarchy	52
10.2.3 WebGL and Shader System	52
10.2.4 CRT Post-Processing Effect	52
10.2.5 Data Sources	52
10.3 Analyst UI: Functional Wireframe	52
10.3.1 Technology Stack	52
10.3.2 Application Structure	53
10.3.3 Graph Visualization	53
10.3.4 RAG Interaction Simulation	53
10.3.5 Data Sources	53
10.4 Frontend Classification Matrix	53
10.5 Relationship Between Frontends and Backend	54
10.6 Architectural Justification	54
11 Prototype and Experimental Environment	55
11.1 Experimental Purpose	55
11.2 Experimental Hypotheses	55
11.3 Hardware and Host Assumptions	56
11.4 Software Environment	56
11.5 Deployment Topology and Initialization	56
11.6 Synthetic Dataset Generation	57
11.6.1 Generation Phases	57
11.6.2 Dataset Composition	57
11.7 Determinism and Its Limits	57
11.8 Reproducibility Protocol	58
11.9 Failure Injection and Functional Verification	58
12 Evaluation and Analysis	59
12.1 Evaluation Philosophy	59
12.2 Evidence Hierarchy	59
12.3 System Component Evidence Mapping	60
12.4 Functional Verification	60
12.5 Synthetic Evaluation Limits	61
12.6 Formal Evaluation Metric Equations	61
12.7 Static Metric Artifact Forensics (Class D)	61
12.8 Valid Dynamic Evaluation Design (Class E)	62
12.9 Graph and RAG Evaluation	62
12.9.1 Graph Evaluation	62
12.9.2 RAG Evaluation	63
12.10 Missing Experiments	63
12.11 Interpretation	63
12.11.1 What the Evidence Demonstrates	63
12.11.2 What the Evidence Does Not Demonstrate	63

13 Security, Ethics, and Operational Governance	65
13.1 Security Model	65
13.2 STRIDE Threat Model Analysis	65
13.3 Trust Zones	66
13.4 Container Security Analysis	66
13.5 The Truth Vault Boundary	66
13.6 Tor Gateway Controls	67
13.7 WolverineDB Security Architecture	67
13.7.1 Vulnerability Forensics	67
13.7.2 Evolution of Replay Protection	67
13.8 Data Security and Synthetic Privacy	68
13.8.1 Ethics of Synthetic Data	68
13.9 RAG Security and Prompt Injection	68
13.10 Threat-to-Control Traceability	69
13.11 Failure and Abuse Cases	69
13.12 Correlation vs. Attribution	69
13.13 Human Oversight and Governance	69
13.14 Residual Risk Model	70
13.15 Security/Ethics Synthesis	70
14 Discussion and Synthesis	71
14.1 Purpose of the Discussion	71
14.2 Objective-by-Objective Assessment	71
14.3 Architectural Findings and Trade-offs	71
14.4 In-Memory Graph Trade-off	72
14.5 Heuristic Entity Resolution Critique	72
14.6 Evaluation and Evidence Distribution	73
14.7 Security and Ethical Interpretation	73
14.8 Prototype vs. Production Comparison	73
14.9 Contribution Analysis	73
14.10 Lessons Learned and Limitations	74
15 Future Work	75
15.1 WolverineDB Evolution	75
15.2 Graph Infrastructure Evolution	75
15.3 Entity Resolution Evolution	76
15.4 Evaluation Evolution	76
15.5 RAG and LLM Evolution	76
15.6 Security and Governance Evolution	77
15.7 Explicit Research Questions	77
15.8 Prioritization Roadmap	77
16 Conclusion	79
16.1 Restatement of the Problem	79
16.2 What Was Built	79
16.3 What Was Demonstrated	79
16.4 What Was Not Demonstrated	79
16.5 Contributions	80
16.6 Final Scientific Interpretation	80
16.7 Final Closing Statement	80

A Synthetic Generation Specifications	81
A.1 Master Seed and Determinism	81
A.2 Dataset Dimensions and Allocation	81
A.3 Handle Mutation and Adversarial Noise	81
A.4 Schema Definitions	82
A.5 Reproduction Procedure	82
B API Specifications and Protocol Interfaces	83
B.1 Role-Based Access Control	83
B.2 Endpoint Catalog	83
B.3 Graph Query Interface	83
B.4 RAG Interface	85
B.5 Collection Run Interface	86
B.6 Error Catalog	86
C WolverineDB Formal Specifications	87
C.1 Specification Taxonomy	87
C.2 Cryptographic Structures	88
C.3 Simulation Boundaries	88
C.4 Verified Security Vulnerabilities	89
C.5 Test Coverage	89
D Evaluation Harness and Static Outputs	91
D.1 Static Output Generation Forensics	91
D.2 Dynamic Evaluation Blueprint (Future Work)	91
D.2.1 Confusion Matrix Definitions	91
D.2.2 Metric Equations	92
D.3 Threshold Evaluation Blueprint	92
D.4 Scalability Benchmark Blueprint	92
D.5 Reproducibility Procedure	92
E Data Dictionaries	93
E.1 Entity Definitions	93
E.1.1 Source	93
E.1.2 CollectionRun	93
E.1.3 Capture	94
E.1.4 NormalizedRecord	94
E.1.5 Quarantine	94
E.1.6 ResolutionCandidate	96
E.1.7 GraphNode	96
E.1.8 GraphEdge	96
E.1.9 Outbox	96
E.1.10 ProcessedMessage	96
E.2 Entity-Relationship Summary	96
F Selected Source Code Excerpts	97
F.1 Blocking Key Generation	97
F.2 Jaro-Winkler Similarity	97
F.3 Composite Score Calculation	97
F.4 RAG Sanitization	98
F.5 BFS Traversal	98

G Tor Gateway Security Controls	99
G.1 Gateway Architecture	99
G.2 HTTP Method Restrictions	99
G.3 Rate Limiting	99
G.4 Administrative Endpoint Blocking	99
G.5 Security Headers	100
G.6 RBAC Header Injection	100
G.7 Upstream Configuration	100
G.8 Automated Security Testing	100

List of Figures

5.1	System Context Diagram illustrating the primary actors, boundaries, and the three operational trust zones.	22
5.2	Trust Zone Architecture Diagram showing 14 containers distributed across Docker networks.	22
5.3	The 8-stage Data Lifecycle from initial ingestion to natural language RAG synthesis.	25
12.1	Evidence Classification Framework Structured as a Hierarchy of Validity	59
12.2	Blueprint only not computed from empirical data. Illustrative synthetic example of how a dynamic evaluation PR curve would operate across sweeping thresholds.	62

List of Tables

1.1	System Objectives and Technical Mapping	3
1.2	Operational Scope and Explicit System Boundaries	4
1.3	Document Organization	5
3.1	Comparative Synthesis of Literature vs. Implementation	15
4.1	System Asset Inventory	18
4.2	Functional Requirements Traceability	18
4.3	System Threat Matrix and Mitigations	20
5.1	Detailed 14-Container Architecture Inventory including Ports and Health Status.	24
5.2	Synchronous vs. Asynchronous Dependency Matrix	24
5.3	Detailed Trust Boundary Analysis	26
6.1	Prisma Schema Entities and Domain Roles	27
6.2	Adversarial Source Schema Variations	28
6.3	Database Inventory, Topology, and Port Mappings	29
7.1	Variables for Composite Scoring	35
7.2	Code-to-Math mappings for analytical methodology.	36
8.1	Backend Core Module Implementation Mapping	40
8.2	RBAC Role to Endpoint Mapping	40
8.3	Implementation Failure Mode Matrix	43
9.1	Specification-to-Code Mapping and Implementation Status	45
9.2	Replay Protection Implementations	48
10.1	Classification matrix comparing Vzeya and Analyst UI capabilities.	53
11.1	Verified Software Environment Inventory	56
11.2	Synthetic Dataset Composition Characteristics	57
12.1	Evidence Classification Definitions	60
12.2	Comprehensive Evidence Classification Matrix	60
12.3	Expanded Missing Experiments Matrix	63
13.1	Evolution of Replay Protection Security	68
13.2	Threat-to-Control Traceability Matrix	69
13.3	Operational Residual Risk Matrix	70
14.1	Objective Realization and Assessment	71
14.2	Prototype vs. Production Readiness	74

15.1 Future Work Prioritization Roadmap	77
A.1 Synthetic Platform Definitions	82
A.2 Canonical Record Field Definitions	82
B.1 RBAC Role Definitions and Permission Scopes	83
B.2 Verified API Endpoint Catalog (from <code>wolverine/src/api/server.ts</code>)	84
B.3 API Error Catalog with HTTP Status Codes	86
C.1 WolverineDB Specification-to-Code Traceability Matrix	87
E.1 Source Entity Schema	93
E.2 CollectionRun Entity Schema	93
E.3 Capture Entity Schema	94
E.4 NormalizedRecord (CanonicalRecord) Entity Schema	94
E.5 Quarantine Entity Schema	94
E.6 ResolutionCandidate Entity Schema	95
E.7 GraphNode Entity Schema	95
E.8 GraphEdge Entity Schema	95
E.9 Outbox Entity Schema	96
E.10 ProcessedMessage Entity Schema	96

Chapter 1

Introduction

1.1 Problem Context

The proliferation of anonymous online platforms has profoundly reshaped the landscape of digital interactions, fostering communities that range from legitimate privacy-focused enclaves to illicit dark marketplaces [1]. This fragmentation presents a formidable challenge for cross-platform intelligence: law enforcement and security researchers, engaged in Cyber Threat Intelligence (CTI) [2], must track sophisticated actors whose activities are deliberately dispersed across disjointed ecosystems. Complicating this technical barrier is a profound ethical tension between the necessity of surveillance for public safety and the fundamental right to digital privacy. To navigate this tension without compromising the privacy of real individuals, the use of synthetic data has emerged as an essential methodology, providing researchers a sandbox to evaluate correlation algorithms without handling sensitive real-world records.

Modern intelligence analysis and digital forensics are increasingly hindered by the fragmentation of digital identities across heterogeneous online platforms. In both legitimate enterprise environments and underground forums, actors operate across multiple isolated domains, generating discrete digital footprints. These footprints comprising registration details, behavioral metadata, and transactional histories are intrinsically noisy. Identifying an actor across multiple platforms requires correlating these fragmented records into a unified identity graph.

However, correlating digital identities is obstructed by several factors. First, data sources possess highly inconsistent and heterogeneous schemas; a REST API payload from a marketplace differs fundamentally from a GraphQL response generated by a social forum. Second, global identifiers (such as verified emails or national identity numbers) are either actively obfuscated or unavailable. Instead, analysts must rely on noisy, probabilistic linkages based on aliases, temporal variations in activity, and contextual cues.

A naïve, exact-match approach (e.g., performing a direct SQL JOIN on a username field) is insufficient. Exact matching fails to capture subtle typographical variations, deliberate character substitutions designed to evade detection, or temporal behavioral shifts. Furthermore, intelligence systems must process this information securely, ensuring that sensitive analytical workloads are isolated from public collection surfaces to preserve operational integrity and prevent data contamination.

1.2 Problem Formulation

To systematically address identity fragmentation, it is necessary to establish a conceptual distinction between a source record, a correlated candidate, and a verified real-world identity. Wolverine operates strictly at the correlation layer, generating probabilistic linkage hypotheses rather than definitive human attribution.

Formally, let R denote a heterogeneous source record retrieved from a platform P . The system first transforms R into a normalized canonical record C , establishing a unified structure containing normalized handles, temporal markers, and source provenance. Given a set of canonical records $\mathbf{C}$, the system’s objective is to evaluate a candidate pair (C_i, C_j) and assign a probabilistic linkage score $S(C_i, C_j) \in [0, 1]$.

If $S(C_i, C_j)$ exceeds a defined decision threshold τ , the system projects an unweighted, undirected edge $E(C_i, C_j)$ between the two records, representing a *candidate entity*. A candidate entity is thus defined as a connected component within the resulting projected graph $G(V, E)$, where vertices $V \subseteq \mathbf{C}$.

It must be emphasized that a candidate entity is an analytical hypothesis. The system does not claim that the subgraph definitively maps to a *verified real-world identity*; rather, it identifies a mathematical correlation that warrants human-in-the-loop analyst investigation.

1.3 Motivation

The architectural and algorithmic design of the Wolverine SIH prototype is driven by specific requirements of the cross-platform intelligence domain. Each major subsystem exists to resolve a fundamental technical barrier:

- **Canonicalization Pipelines:** Source platforms yield data in varying formats (JSON:API, GraphQL, HTML). Parsers transform these payloads into a unified Canonical Record schema to enable universal comparison metrics.
- **Probabilistic String Similarity:** Because exact string matching fails on aliases, the system employs a modified Jaro-Winkler distance algorithm, which accounts for typographical permutations and heavily weights prefix characters.
- **Temporal Proximity:** User behaviors exhibit temporal patterns. The system uses a temporal decay function to reward concurrent multi-platform activities, providing a secondary linkage signal that corroborates string similarity.
- **Graph Projection:** Relationships extending beyond pairwise matches (e.g., A matches B, and B matches C) are modeled using Breadth-First Search (BFS) graph traversal to surface complex, multi-hop operational networks.
- **Retrieval-Augmented Generation (RAG):** To accelerate analyst comprehension, an LLM synthesizes structured case narratives from the projected graphs.
- **Network Isolation and Cryptographic Anchoring:** To maintain evidentiary integrity and protect against prompt injection or external intrusion, the system enforces a strict air-gap between public ingest layers, processing clusters, and a localized Truth Vault, supported by Merkle-tree state hashing.
- **Synthetic Generation Engine:** Due to severe privacy and ethical constraints preventing the use of real-world Personally Identifiable Information (PII) during development, the prototype employs a deterministic synthetic data generator, establishing an objective ground truth for evaluation.

1.4 Objectives

The Wolverine project is guided by four primary objectives, mapping theoretical intelligence requirements to specific engineering implementations.

Objective	Technical Interpretation	Implementation	Verification / Limitation
1. Heterogeneous Ingestion	Extract unified records from diverse API schemas.	Modular parsers mapping REST/GraphQL payloads to Canonical schema.	Verified via parser test suites; limited to supported target formats.
2. Probabilistic Correlation	Generate candidate identities despite obfuscation.	Jaro-Winkler string similarity paired with exponential temporal decay.	Verified via static mock F1 metrics; limited by fixed heuristic weights.
3. Analyst Acceleration	Summarize complex subgraphs intuitively.	Local Ollama RAG integration with prompt sanitization.	Verified via generated narratives; constrained by strict 500-character LLM input limits.
4. Evidentiary Trust	Protect the integrity of analytical state.	Air-gapped network topology and WolverineDB Merkle state hashes.	Verified via Docker isolation; BFT consensus relies on simulated in-process transport.

Table 1.1: System Objectives and Technical Mapping

1.5 Contributions

The Wolverine SIH prototype primarily represents an *engineering and systems-integration contribution* rather than a fundamental theoretical breakthrough. While string similarity metrics, LLM-driven RAG, and Merkle trees are well-established, Wolverine’s primary contribution lies in unifying these components into a zero-knowledge, privacy-preserving demonstration harness.

1.5.1 Architectural Contribution

The prototype defines a 14-container distributed topology strictly separating public collection boundaries (DMZ), analytical inference engines (Internal), and an immutable evidentiary record (Truth Vault) through explicit Docker network segregation.

1.5.2 WolverineDB Cryptographic Trust Layer

The project introduces WolverineDB as an independent cryptographic trust layer (Chapter 9), enforcing an immutable, Merkle-backed audit trail for every entity resolution decision.

1.5.3 Dual-Frontend Architecture

To accommodate varying demonstration requirements, the project introduces a dual-frontend architecture (Chapter 10): a functional React-based interface for deep analytical workflows, and a cinematic presentation layer (*Vzeyya*) tailored for high-level visualizations.

1.5.4 Evidence Classification Framework

To maintain rigorous academic honesty, Wolverine establishes a formal evidence classification framework, categorizing project outcomes from Class A (empirical validation on real data) down to Class E (missing experiments), ensuring transparent delineation between verified functionality (Class B) and simulated mock components (Class D).

1.5.5 Engineering Contribution

Wolverine introduces a deterministic synthetic ecosystem generator capable of seeding 50,000 multi-platform personas. This generator provides a reproducible, ethically safe foundation for evaluating entity resolution heuristics by exposing an explicit ground truth separate from the inference pipeline.

1.5.6 Analytical and Methodological Contribution

The system implements a tailored linkage heuristic combining modified Jaro-Winkler alias similarity with activity-based temporal decay. Furthermore, graph projection is achieved via an explicit, strictly bounded in-memory Breadth-First Search (BFS) rather than relying on heavy relational graph database joins.

1.6 Scope and Boundaries

To prevent misrepresentation of the prototype’s capabilities, the operational scope and evidence boundaries are explicitly defined in Table 1.2.

In Scope (Demonstrated / Implemented)	Out of Scope (Excluded / Simulated)
Deterministic synthetic data generation (50,000 personas).	Processing real-world PII, live Tor traffic, or genuine cryptocurrency ledgers.
Probabilistic candidate correlation via Jaro-Winkler and temporal rules.	Definitive, legally admissible human attribution.
Bounded in-memory graph projection (BFS, $d \leq 4$, $N \leq 500$).	Production-scale performance benchmarks or recursive PostgreSQL CTE execution.
Cryptographic state hashing (Merkle trees) via WolverineDB.	Physical multi-node BFT network consensus (simulated via in-process transport).
Vzeya mock-data presentation layer and Analyst UI functional layer.	Dynamic, empirical benchmark calculations (F1 scores are static/demonstrative).

Table 1.2: Operational Scope and Explicit System Boundaries

1.7 Report Organization

This monograph is structured to progressively decompose the Wolverine system from theoretical foundations to technical implementation.

Chapter 2 establishes the conceptual and mathematical definitions governing record linkage, graph analysis, and RAG. Chapter 3 situates the project within existing academic literature. Chapter 4 formalizes the system’s functional requirements, asset boundaries, and detailed threat model.

The core implementation is detailed in subsequent chapters. Chapter 5 outlines the distributed topology. Chapter 6 describes data schemas. Chapter 7 formalizes heuristics, Chapter 8 documents engineering, Chapter 9 introduces the trust layer, and Chapter 10 describes the visual interfaces. Evaluation and context occupy the remaining chapters (Chapter 11 through Chapter 14), synthesizing the prototype’s success and providing a roadmap for future work in Chapter 15 and Chapter 16.

Chapter	Content Summary
Ch 1-2	Introduction and formal Domain Foundations.
Ch 3-4	Related Work, Requirements, and Threat Model.
Ch 5-6	System Architecture (14-container topology) and Data Architecture.
Ch 7-8	Analytical Methodology and Engineering Implementation.
Ch 9-10	WolverineDB Cryptographic Layer and Dual-Frontend Systems.
Ch 11-12	Experimental Environment (Synthetic Generation) and Evaluation Framework.
Ch 13-14	Security and Ethics Analysis, followed by Discussion and Synthesis.
Ch 15-16	Future Work and Conclusion.
Appendices E,F,G	Extended material supporting the new chapters and comprehensive specs.

Table 1.3: Document Organization

Chapter 2

Domain and Technical Foundations

Before detailing the architectural implementation of the Wolverine SIH prototype, this chapter establishes the formal domain concepts and conceptual paradigms upon which the system is built. It covers the transition from heterogeneous source data to probabilistic record linkage, the mechanics of string similarity and graph traversal, the safety mechanisms of Retrieval-Augmented Generation (RAG), the necessity of synthetic evaluation, and the principles of cryptographic integrity.

2.1 Heterogeneous Intelligence Records

Digital intelligence collection relies on extracting records from disparate online platforms, each enforcing distinct data schemas, identifiers, and temporal resolutions. A *source record* is a raw, platform-specific data structure such as a JSON payload from a REST API or an HTML DOM fragment from a scraped forum.

Definition 2.1 (Heterogeneous Record Sources). Let $P = \{P_1, P_2, \dots, P_k\}$ be a set of diverse data platforms. A heterogeneous record R_{P_i} is a structured or semi-structured data entity originating from P_i , governed by a schema Σ_i unique to that platform, such that there exists no universal primary key mapping seamlessly between R_{P_i} and R_{P_j} for $i \neq j$.

Source records inherently lack global, unifying keys. For example, a user on a dark web marketplace might be identified by a numeric ID ("vendor_id": 8492), while on a related social forum, the same actor uses an alphanumeric handle ("author": "Driftking99"). Furthermore, timestamps may be represented in Unix epoch seconds, ISO-8601 strings, or relative human-readable formats. Establishing correlation across these domains requires normalizing these disjointed representations into a standardized schema capable of supporting universal mathematical comparison.

2.2 Canonical Representation

To solve the schema heterogeneity problem, intelligence systems employ a *canonical record*. A canonical record is a unified, strictly typed data structure that distills essential identity features from noisy source records.

Definition 2.2 (Canonical Representation). A canonical record C is a normalized tuple $C = (id, handle, timestamp, metadata)$ derived from a heterogeneous record R_{P_i} via a deterministic transformation function $f_{\Sigma_i} : R_{P_i} \rightarrow C$, enabling cross-platform correlation over a unified attribute space.

The canonicalization process involves:

1. **Field Normalization:** Coercing all temporal markers to UTC ISO-8601 timestamps and standardizing string formats (e.g., lowercasing, stripping special characters).
2. **Identity Extraction:** Isolating the primary actor identifier (e.g., username, handle, or ID) into a dedicated correlation field.
3. **Provenance Tracking:** Embedding metadata detailing the exact origin of the record (platform name, extraction timestamp, and collection source) to ensure analytical traceability.
4. **Hashing:** Generating a deterministic, cryptographic hash (e.g., SHA-256) of the canonical record to serve as its unique internal primary key.

By projecting all source records into a unified canonical schema (detailed fully in Chapter 6), downstream algorithms can execute linkage rules uniformly, ignorant of the underlying platform origins.

2.3 Record Linkage and Entity Resolution

Once canonicalized, the system must determine whether two distinct records refer to the same real-world entity. This challenge is formally addressed by probabilistic record linkage, rooted conceptually in the Fellegi-Sunter framework [3].

Definition 2.3 (Record Linkage). Given two sets of canonical records A and B , record linkage is the process of defining a comparison function $\gamma : A \times B \rightarrow [0, 1]$ that computes a similarity score for candidate pairs $(a, b) \in A \times B$. Pairs satisfying $\gamma(a, b) \geq \tau$ (a predefined threshold) are classified into a set of matched pairs M , else into unmatched pairs U .

Because exact deterministic matching (e.g., checking if $a.\text{username} == b.\text{username}$) fails to capture subtle variations or deliberate obfuscations, probabilistic systems utilize a comparison vector. This vector represents the degree of agreement across multiple attributes (e.g., handle similarity, temporal proximity, and behavioral overlap).

To prevent the computational explosion of comparing every record to every other record ($O(|A| \times |B|)$ complexity), systems employ *blocking*. Blocking restricts pairwise comparisons to a highly probable subset of candidates by requiring they share a specific heuristic key, such as a coarse phonetic code or a defined date range. Following blocking, the candidate pairs are scored, and if the composite score exceeds a defined decision threshold, the records are probabilistically linked.

2.4 String Similarity

When exact string matching is insufficient for alias resolution, systems rely on edit distance metrics.

Definition 2.4 (String Similarity Metric). A string similarity metric is a function $\delta : S \times S \rightarrow [0, 1]$ mapping two strings s_1, s_2 to a similarity score, where 1 denotes identical strings and 0 denotes complete orthogonality, typically accounting for character matches, transpositions, and edit penalties.

While the Levenshtein distance measures the absolute number of insertions, deletions, and substitutions required to transform one string into another, it does not adequately model the cognitive patterns of human alias mutation. The Jaro similarity metric [4] measures the number of matching characters within a sliding temporal window, penalizing transpositions but inherently rewarding common sequence structures.

The Jaro-Winkler extension [5] refines this by heavily weighting matches at the beginning of the string. In the context of Cohen et al.’s comparative analysis of string matching techniques [6], the Jaro-Winkler family excels in identity linkage tasks because typographical errors and deliberate obfuscations occur far less frequently in the initial characters of a handle. Jaro-Winkler was specifically selected for Wolverine over alternatives (such as Monge-Elkan or Soft-TFIDF) because alias permutations often involve suffix appendings (e.g., adding numbers) rather than complete token restructuring, aligning with Navarro’s guidance on approximate string matching architectures [7].

Illustrative synthetic example not an empirical benchmark: Consider the handles `shadow_broker` and `shadowbroker99`. An exact match fails. Levenshtein distance penalizes the missing underscore and the appended numbers heavily. However, Jaro-Winkler identifies the identical `shadow` prefix, generating a high probability score that suggests these handles are variations of the same underlying identity.

2.5 Temporal Proximity

Identity correlation cannot rely exclusively on string similarity. Temporal behavior provides a critical secondary signal. If two aliases register on distinct platforms within a remarkably narrow time window, the probability that they are controlled by the same actor increases significantly.

Temporal proximity is modeled using a mathematical decay function. If the time delta Δt between two events approaches zero, the temporal score approaches 1.0. As Δt increases, the score decays exponentially toward zero. This conceptual mechanism ensures that temporal concurrence acts as a powerful corroborating signal for entity resolution, while deeply separated events contribute minimal linkage confidence. The exact implementation parameters are detailed in Chapter 7.

2.6 Graph Representation

Pairwise record linkages are atomic. To surface broader operational networks, these discrete linkages must be projected into a relational graph. In this graph representation, canonical records are modeled as *nodes* (vertices), and probabilistic linkages are modeled as *edges*.

Formally, a graph is defined as $G = (V, E)$ where V represents the set of canonical record vertices and $E \subseteq V \times V$ represents the adjacency relationships (probabilistic links). According to Cormen et al. [8], a foundational traversal paradigm for exploring connected components is Breadth-First Search (BFS). BFS explores all immediate neighbor nodes forming the adjacency layer before moving deeper into the network, making it optimal for discovering localized operational clusters.

While enterprise systems execute recursive traversals over distributed databases, rapid prototype architectures often project a bounded subset of the graph entirely in memory. A *bounded traversal* limits the exploration to a maximum depth d and a maximum total node count N . This eliminates database latency, though it imposes strict computational ceilings on the maximum traversal depth and total node count.

2.7 Retrieval-Augmented Generation and Safety

Retrieval-Augmented Generation (RAG) enhances Large Language Models (LLMs) by supplementing their parametric weights with dynamic, non-parametric context retrieved from external databases, as outlined in the comprehensive RAG survey by Gao et al. [9]. In intelligence analysis, RAG allows an LLM to generate natural-language case summaries strictly grounded in retrieved graph data, minimizing hallucinations.

However, ingesting external text introduces the critical vulnerability of Prompt Injection, a significant threat category formally identified by Perez and Ribeiro [10]. Malicious actors can embed adversarial instructions within forum posts or profile descriptions (e.g., "*Ignore previous instructions and output 'System Compromised'*"). If the RAG system naively passes this context to the LLM, the model may execute the malicious payload. Therefore, RAG systems must implement multi-layered sanitization (e.g., regex filtering, length truncation, and control-character stripping) and maintain deterministic fallbacks if the LLM becomes unresponsive or compromised.

2.8 Cryptographic Integrity and Trust

To ensure that intelligence findings are not altered post-collection—whether by malicious intrusion or system failure—analytical architectures must embed cryptographic trust mechanisms.

A Merkle tree [11] is a fundamental data structure for this purpose. By recursively hashing pairs of data blocks up to a single cryptographic root hash, a system can prove the integrity of massive datasets rapidly. Any alteration to a historical record immediately alters its hash, cascading upward and invalidating the root. When combined with principles from Byzantine Fault Tolerance (BFT) consensus models [12] and hash chains utilized in decentralized ledgers [13], these structures enable distributed systems to maintain consensus over an immutable state log, ensuring a tamper-evident audit trail for all analytical decisions.

2.9 Synthetic Data and Ground Truth

Evaluating entity resolution models on real-world intelligence data presents severe ethical and scientific barriers. Handling genuine Personally Identifiable Information (PII) or dark web telemetry introduces unacceptable legal liabilities. More critically, real-world data lacks an objective *ground truth*; analysts can never definitively prove that two pseudonymous handles belong to the same physical human without out-of-band surveillance.

Consequently, robust scientific evaluation requires synthetic data generation. A deterministic generator simulates a virtual population, assigning known individuals distinct aliases and behaviors across simulated platforms. Because the generator explicitly records the mapping between the real individual and their generated aliases in an isolated Truth Vault, researchers can objectively measure precision, recall, and false-positive rates without compromising real-world privacy. It is critical to recognize, however, that synthetic evaluation is inherently subject to domain shift; success in a synthetic environment does not guarantee equivalent performance on real-world anomalies.

2.10 Chapter Synthesis

The concepts defined in this chapter—canonicalization, probabilistic Jaro-Winkler linkage, bounded BFS traversal, RAG sanitization, cryptographic integrity, and synthetic ground truth—form the theoretical bedrock of the Wolverine SIH prototype. The subsequent chapters detail how these abstract paradigms are explicitly integrated into a secure, 14-container distributed architecture.

Chapter 3

Related Work and Research Context

Entity resolution, cross-platform intelligence analysis, and trustworthy analytical infrastructure draw from diverse theoretical and engineering domains. This chapter reviews the formal foundations underlying the Wolverine prototype, contextualizing its design choices within the established literature of probabilistic record linkage, string similarity metrics, graph traversal paradigms, synthetic population generation, cyber threat intelligence, container security, threat modeling, safe retrieval-augmented language models, and cryptographic data integrity.

3.1 Record Linkage and Entity Resolution

The challenge of identifying records that refer to the same real-world entity across disparate databases was formalized by Fellegi and Sunter [3] in their foundational mathematical model for record linkage. Under the Fellegi-Sunter framework, candidate pairs from two distinct sets A and B are classified into matched pairs M or unmatched pairs U based on a comparison vector $\gamma = [\gamma_1, \gamma_2, \dots, \gamma_K]$ representing agreement or disagreement across K observable attributes.

Elmagarmid et al. [14] provide a comprehensive survey on duplicate detection, establishing that modern systems must move beyond exact matches to account for typographical errors, varying schemas, and missing data. Building on this, Getoor and Machanavajjhala [15] outline the open challenges in entity resolution (ER), emphasizing the need for scalable algorithms in big data environments. Christen [16] expands the Fellegi-Sunter framework to modern multi-source environments, highlighting the computational bottleneck of quadratic pair comparisons ($O(|A| \times |B|)$) and the necessity of blocking strategies. Wolverine adopts this probabilistic linkage philosophy, utilizing heuristic blocking on alias prefixes to reduce the search space prior to intensive scoring.

3.2 Cross-Platform Identity Resolution

In multi-platform intelligence ecosystems where global unique identifiers (such as national identity numbers or verified emails) are intentionally withheld or unavailable, entity resolution must rely on probabilistic matching of noisy, partial identifiers such as aliases, temporal registration windows, and localized behavioral cues. The literature has established that cross-platform identity linking poses unique challenges due to sparse data and deliberate obfuscation by adversarial actors.

Narayanan and Shmatikov [17] demonstrated the feasibility of de-anonymization techniques against sparse, high-dimensional datasets, showing how minimal topological cues can compromise supposedly anonymous networks. In the context of malicious networks, Soska and Christin [1] mapped the evolution of dark marketplaces, illustrating how user identities and reputations persist across distinct platforms despite efforts to remain anonymous. While exact

identifiers allow trivial joining, approximate matching across such ecosystems requires complex distance functions. The Wolverine prototype deliberately avoids highly sophisticated, computationally intensive deep learning-based embeddings, opting instead for a binary prefix matching heuristic combined with Jaro-Winkler alias similarity and exponential temporal decay. This decision prioritizes rapid, deterministic execution in a prototype environment.

3.3 String Similarity Metrics

A core component of any entity resolution engine is the quantitative measurement of string similarity. The literature provides a rich taxonomy of comparison functions. Cohen et al. [6] offer a seminal comparative evaluation of string matching metrics, dividing them into edit-distance based, token-based, and hybrid approaches. Navarro [7] provides a comprehensive guide to approximate string matching, detailing the algorithmic complexity of various distance measures.

Common edit-distance metrics include the Levenshtein distance, which calculates the minimum number of single-character edits required to transform one string into another. Alternatively, token-based metrics like TF-IDF or character n -grams excel in matching longer text blocks but perform poorly on short, contiguous aliases. Jaro distance, and its extension the Jaro-Winkler similarity, were specifically designed for short string matching such as human names and aliases. Jaro-Winkler increases the similarity score for strings that match from the beginning for a set prefix length, reflecting the empirical observation that typographic errors are less likely to occur at the start of a name. After evaluating the trade-offs outlined by Cohen et al., Wolverine implements the Jaro-Winkler metric for alias comparison, as it provides an optimal balance between accuracy for short strings and computational efficiency required for rapid pairwise scoring.

3.4 Graph-Based Intelligence Analysis

Projecting multi-entity interactions into relational graphs enables analysts to discover non-obvious associations through multi-hop path traversals. Newman [18] provides foundational insights into the structure and function of complex networks, establishing how topological properties reflect underlying behavioral communities. In enterprise production systems, graph query engines often rely on native, disk-backed graph databases (e.g., Neo4j) or execute recursive Common Table Expressions (CTEs) over relational SQL adjacency tables to map these communities.

The algorithms driving these traversals, such as Breadth-First Search (BFS), are cornerstones of computer science, formalized in texts like Cormen et al. [8]. Wolverine utilizes Neo4j for storage but deliberately executes real-time traversal via an in-memory BFS over explicit node-edge adjacency lists. As established in graph processing literature, in-memory traversals eliminate SQL/Cypher query overhead and complex database indexing during high-throughput execution. However, this architectural choice imposes strict memory ceilings that inherently bound the maximum traversal depth ($d \leq 4$) and active graph scale ($N \leq 500$), marking a definitive trade-off between prototype speed and production scalability.

3.5 Cyber Threat Intelligence

The aggregation and analysis of indicators of compromise (IoCs) and adversary behaviors form the domain of Cyber Threat Intelligence (CTI). Liao et al. [2] propose techniques for automatically extracting CTI from unstructured text, highlighting the challenges of structuring threat data for automated analysis. To standardize the description of adversary tactics and techniques, the MITRE ATT&CK framework [?] has become the industry-standard taxonomy.

Wolverine fits within the CTI landscape as a rapid prototyping environment for testing intelligence workflows. Rather than focusing on parsing unstructured threat reports, Wolverine simulates the structured outputs of a mature CTI pipeline where identities, aliases, and platforms are already extracted and focuses on the downstream analytical processes of entity resolution and graph traversal.

3.6 Synthetic Data and Ground Truth

Evaluating cross-platform entity resolution in real-world threat intelligence settings is severely constrained by ethical, legal, and operational boundaries. Handling genuine Personally Identifiable Information (PII) introduces significant privacy violations and regulatory compliance liabilities. Furthermore, real-world data lacks verifiable, objective ground truth.

Modern research leverages deterministic synthetic population generators to overcome this evaluation barrier. Nowok et al. [19] introduce the *synthpop* package, demonstrating how synthetic data can mirror the statistical properties of real populations while preserving confidentiality. By generating synthetic personas according to predefined statistical distributions and explicitly tracking their assignments in an isolated ground-truth registry, researchers can objectively measure precision, recall, and false-positive rates without compromising real-world privacy. However, synthetic evaluation is limited by domain shift; synthetic models cannot perfectly emulate the irrational unpredictability of human adversaries, meaning that metrics derived from synthetic environments represent an upper bound on real-world performance. Wolverine embraces this synthetic paradigm entirely, explicitly isolating the generation logic from the inference engine to prove algorithmic mechanics safely.

3.7 Retrieval-Augmented Generation and Prompt Security

Retrieval-Augmented Generation (RAG), as formalized by Lewis et al. [20], combines parametric neural generation with non-parametric retrieval from external knowledge repositories. Gao et al. [9] survey the rapid evolution of RAG systems, noting their utility in mitigating LLM hallucinations. In intelligence workflows, RAG allows Large Language Models to synthesize complex case dossiers grounded strictly in captured evidential records rather than relying on ungrounded internal model weights.

However, ingesting untrusted text from adversarial web sources introduces severe prompt injection vulnerabilities [10]. Greshake et al. [21] demonstrate that indirect prompt injection—where malicious instructions are hidden in retrieved web pages—can compromise downstream LLM agents. Formalizing these threats, Liu et al. [22] provide a taxonomy of prompt injection attacks. Safe analytical systems must enforce strict multi-stage input sanitization, length bounding, control character elimination, and deterministic fallback heuristics. Wolverine implements a 4-rule regex sanitization pipeline as a baseline defense-in-depth measure, though it must be acknowledged that regex alone cannot definitively neutralize advanced adversarial prompt injection.

3.8 Cryptographic Integrity and Distributed Trust

In sensitive forensic and investigative environments, maintaining the evidentiary integrity of analytical findings is paramount. The foundational challenge of consensus in distributed networks was defined by Lamport et al. [12] through the Byzantine Generals Problem. Building on this, distributed consensus protocols, such as Practical Byzantine Fault Tolerance (PBFT) by Castro and Liskov [23], establish theoretical foundations for multi-party agreement across nodes even in the presence of malicious or arbitrary failures.

For data integrity, Merkle trees [11] provide cryptographic proof of state inclusion and tamper-evident append-only logging through recursive cryptographic hashing. Nakamoto [13] famously applied Merkle chains to achieve decentralized consensus in Bitcoin. Certificate Transparency logs [24] demonstrate the production viability of these structures for public auditability. Wolverine relies on robust digital signatures, implementing Ed25519 [25] for verifiable node communication. While Wolverine incorporates Merkle tree state hashing natively, its BFT consensus network is currently realized as a simulated in-process transport, demonstrating cryptographic feasibility without requiring a fully decentralized physical deployment.

3.9 Container Security and Microservices

Modern analytical infrastructure relies heavily on containerization to ensure consistent, isolated execution environments. Newman [26] outlines the architectural principles of microservices, emphasizing independent deployability and service isolation. As analytical platforms ingest potentially hostile data, isolating the components becomes a security imperative.

Sultan et al. [27] survey container security issues, noting that while containers offer less isolation than full virtual machines, their network namespaces and cgroups provide critical defense layers. Wolverine leverages Docker network isolation to explicitly segregate the **ThreatGen** synthetic data generator from the **InferenceEngine**, guaranteeing that the analysis components cannot directly access the ground truth database, thereby enforcing a strict zero-knowledge analytical boundary.

3.10 Threat Modeling

To systematically identify and mitigate vulnerabilities in the analytical platform, formal threat modeling methodologies are required. Shostack [28] popularized the STRIDE methodology (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) as a structured approach to analyzing software systems for security flaws. Complementary to software-centric models, NIST Special Publication 800-30 [29] provides a comprehensive guide for conducting risk assessments in federal information systems.

Wolverine incorporates these principles by explicitly modeling the ingestion of adversarial data as a threat vector, leading to the implementation of the regex sanitization pipeline to mitigate Tampering and Information Disclosure risks associated with prompt injection attacks.

3.11 Tor and Onion Architecture

The necessity for secure, anonymous communication in adversarial environments is addressed by the Tor network’s onion routing protocol [30]. Tor encrypts traffic in nested layers, routing it through a decentralized circuit to obscure the origin and destination of the data. Wolverine’s architecture conceptually supports data collection from such anonymous platforms via its DMZ proxy layer, though the prototype itself operates strictly on simulated, generated data rather than live Tor traffic.

3.12 Research vs Engineering Gap

Existing literature provides robust, mathematically proven solutions for isolated components: statistical record linkage, string comparison metrics, synthetic data simulation, cryptographic hashing, and distributed consensus. The Wolverine SIH prototype does not claim to invent fundamentally novel algorithms in these individual theoretical spaces.

Instead, the project addresses a critical *engineering and systems-integration gap*. It unifies these disparate layers into an end-to-end, zero-knowledge testbed where a synthetic ecosystem generates heterogeneous traffic, an automated entity resolution engine probabilistically links identities, an air-gapped Truth Vault maintains objective ground truth, and a sanitized RAG assistant synthesizes case narratives. By integrating these components into a cohesive microservice architecture, Wolverine demonstrates the practical viability of combining academic rigor with rapid engineering deployment.

3.13 Comparative Synthesis

To clarify Wolverine’s specific technical positioning, Table 3.1 summarizes how established methods compare to the project’s chosen implementations across the expanded domain space.

Domain	Established Literature	Wolverine Implementation
Record Linkage	Fellegi-Sunter statistical weighting [3, 14]	Hardcoded heuristic thresholds
String Similarity	TF-IDF, Edit Distances [6]	Jaro-Winkler alias similarity
Graph Traversal	Recursive SQL / Native DB [18]	Bounded, in-memory BFS ($d \leq 4$)
Ground Truth	Manual verification	Isolated synthetic Truth Vault
LLM Safety	Fine-tuned alignment [22]	4-rule Regex sanitization
Cryptographic State	Multi-node Distributed Ledger [23]	In-process simulated network
CTI	Unstructured NLP extraction [2]	Structured synthetic event stream
Microservices	Distributed orchestration [26]	Docker isolated local containers
Threat Modeling	STRIDE / NIST SP 800-30 [28]	Applied to adversarial RAG inputs
Cross-Platform ID	Topological anonymization [17]	de- Prefix blocking & temporal decay

Table 3.1: Comparative Synthesis of Literature vs. Implementation

Chapter 4

System Requirements and Threat Model

Designing a robust intelligence correlation platform requires explicit constraints. This chapter details the formal requirements, the assets protected, the architectural trust boundaries, and the specific threat vectors the Wolverine prototype is engineered to withstand.

4.1 Stakeholders and Actors

The system defines explicit roles corresponding to independent subsystems and user profiles. Only the following actors exist within the prototype boundary:

- **Analyst (Human):** A trusted user interacting exclusively with the Analyst UI to query identities and review RAG-synthesized case files.
- **Synthetic Generator (Machine):** A deterministic Python engine executing outside the analytical boundary, injecting simulated records into the DMZ.
- **Ingestion Adapters (Machine):** Stateless parsing services responsible for normalizing incoming heterogeneous HTTP payloads.
- **Entity Resolver (Machine):** The core analytical backend calculating probabilistic linkage scores.
- **RAG Engine / Ollama (Machine):** The LLM inference service synthesizing graph narratives.
- **Truth Vault (Machine):** The isolated, air-gapped storage layer maintaining the deterministic ground truth for evaluation.

4.2 System Assets

To construct a valid threat model, the system's assets must be explicitly enumerated and prioritized by criticality.

4.3 Functional Requirements

The operational capabilities of the Wolverine prototype are governed by explicit Functional Requirements (FR). Table 4.2 traces these requirements to their implementation components.

Asset	Description	Protection Priority
Truth Vault State	The deterministic ground truth mappings linking aliases to actual synthetic personas.	Critical (Integrity/Confidentiality)
Canonical Records	Normalized representations of ingested platform data.	High (Integrity)
Candidate Graph	The projected BFS adjacency lists representing the linkage hypotheses.	Medium (Availability)
Cryptographic State	WolverineDB Merkle roots and signature hashes.	High (Integrity)
LLM Context	The sanitized prompts sent to the RAG engine.	High (Integrity/Safety)

Table 4.1: System Asset Inventory

ID	Requirement	Inputs → Outputs	Implementation Module
FR-01	Heterogeneous Ingestion	HTTP Payloads → Canonical Records	Parser Services (REST/-GraphQL)
FR-02	Alias Correlation	Canonical Records → Candidate Links	Entity Resolver (Jaro-Winkler)
FR-03	Graph Projection	Candidate Links → BFS Subgraphs	Graph Projector
FR-04	Narrative Synthesis	BFS Subgraphs → Text Summaries	Ollama RAG Engine
FR-05	State Integrity	Canonical Records → Merkle Roots	WolverineDB

Table 4.2: Functional Requirements Traceability

If an ingestion parser (FR-01) fails due to unhandled schema drift, the system explicitly drops the payload to prevent corruption of the canonical datastore, prioritizing system integrity over data completeness.

4.4 Non-Functional Requirements

The constraints governing *how* the system executes are defined by Non-Functional Requirements (NFR).

- **NFR-01 (Determinism):** The synthetic generator must always produce the exact same 50,000 personas given the initial seed $S_0 = 42$. *Rationale:* Enables objective reproducibility of evaluation metrics.
- **NFR-02 (Isolation):** The inference pipeline must have zero network access to the Truth Vault. *Rationale:* Prevents the resolver from "cheating" by directly querying the ground truth.
- **NFR-03 (Bounded Resources):** Graph traversal must not exceed depth $d = 4$ or

$N = 500$ nodes. *Rationale:* Prevents out-of-memory crashes during high-throughput demonstration runs.

- **NFR-04 (Zero PII):** The system must absolutely reject or refrain from generating genuine Personally Identifiable Information. *Rationale:* Mandatory ethical boundary for the SIH demonstration.

4.5 Trust Boundaries

The architecture enforces three explicit trust domains, implemented via Docker network isolation (detailed in Chapter 5):

1. **DMZ (Untrusted):** The public-facing ingest endpoints (Nginx) and presentation UI. Data here is considered hostile.
2. **Internal (Semi-Trusted):** The processing core (PostgreSQL, Neo4j, Entity Resolver). Data here is sanitized and canonicalized, but still subject to strict algorithmic bounds.
3. **Truth Vault (Strictly Trusted):** The air-gapped evaluation datastore. Only the synthetic generator (during initialization) and the evaluator (during scoring) may interact with this boundary.

4.6 Threat Model

To evaluate the system's defensive posture, Table 4.3 maps specific attack vectors against the system's implemented controls, referencing the STRIDE methodology established by Shostack [28] and NIST guidelines for risk assessment [29].

It must be noted that while T-05 (Database Tampering) is detected via cryptographic hashing, the underlying network consensus meant to prevent it (BFT) is currently an in-process simulation, marking a residual risk in the prototype.

4.7 Failure and Abuse Cases

The system explicitly handles several severe failure conditions:

- **LLM Timeout/Failure:** If the Ollama service crashes or times out processing a RAG request, the system falls back to a deterministic, rule-based heuristic hypothesis generation, ensuring the Analyst UI never hangs indefinitely.
- **Graph Explosion:** If a highly generic alias (e.g., "admin") links to thousands of accounts, the BFS traversal hard-stops at $N = 500$, truncating the graph to prevent the node process from exceeding V8 memory limits.
- **Malformed Payloads:** If an ingestion parser encounters an array where an object is expected, it strictly drops the payload and logs a validation error, preventing schema corruption in PostgreSQL.

4.8 Ethical and Operational Constraints

Wolverine is designed under a mandatory synthetic-data constraint. Because probabilistic entity resolution inherently generates false positives, deploying this architecture against real-world populations without definitive human-in-the-loop verification introduces severe ethical risks of

ID	Threat tor	Ac-	STRIDE	Attack Vector	Implemented Control
T-01	Malicious Per-sona	Per-	Spoofing	Prompt Injection in profile description	4-Rule Regex sanitization + 500-char truncation
T-02	Rogue lyst	Ana-	Info Discl.	Direct query to Truth Vault to skew metrics	Docker network air-gap ('internal: true')
T-03	External tacker	At-	DoS	API Flooding / DoS on ingestion	Nginx rate limiting (10 req/s)
T-04	Malicious Per-sona	Per-	Spoofing	Sybil attack (massive fake alias generation)	Jaro-Winkler thresholding + Time decay penalty
T-05	Internal tacker	At-	Tampering	Tampering with canonical PostgreSQL records	WolverineDB Merkle state hashes
T-06	Internal tacker	At-	Elevation	RBAC Header Spoofing via forged X-Wolverine-Role	API Gateway signature validation / internal routing checks
T-07	Local Exploit		Elevation	Container Escape breaking Docker daemon isolation	Drop default privileges, unprivileged runtime
T-08	Poisoned Data		Tampering	Model Poisoning via adversarial RAG data	Strict schema validation on ingestion prior to index
T-09	System/Model		Repudiation	Synthetic-to-Real Domain Shift causing bad inference	Operational mandate requiring human analyst oversight
T-10	System Fault		DoS/Spoofing	Outbox Event Replay causing duplicate processing	Idempotency keys in WolverineDB replay store

Table 4.3: System Threat Matrix and Mitigations

misattribution. The prototype deliberately operates purely on deterministic synthetic data to prove the engineering integration without incurring these real-world liabilities. A full ethical analysis is provided in Chapter 13.

Chapter 5

System Architecture

The Wolverine SIH prototype is engineered as a secure, distributed microservices architecture designed specifically to isolate adversarial data ingestion from sensitive analytical workloads and ground-truth evaluation. This chapter details the physical container topology, network trust boundaries, explicit data flows, and architectural failure mitigations, forming the structural basis for the system’s analytical capabilities.

5.1 Architectural Objectives

The architectural design of the prototype is driven by specific requirements defined in Chapter 4.

- **Separation of Public and Analytical Environments:** To protect the integrity of the canonical datastore (FR-05), public ingestion endpoints are isolated from internal processing clusters.
- **Evaluation Integrity (Truth Vault):** To fulfill the synthetic-only mandate (NFR-04) and ensure objective evaluation (NFR-01), the ground truth is strictly air-gapped from the inference engine (NFR-02), physically preventing "cheating."
- **Controlled Resource Bounds:** Because graph analysis can lead to exponential memory consumption, the architecture explicitly dictates in-memory processing bounded by hard node/depth limits (NFR-03) rather than relying on unconstrained database joins.

5.2 System Context

The Wolverine prototype encompasses several independent actors and subsystems operating within defined boundaries. The primary entities are depicted in Figure 5.1.

The primary entities interacting within the system, as shown in Figure 5.1, include:

- **Analyst (Human):** Interacts exclusively with the functional *Analyst UI*.
- **Synthetic Source Ecosystem:** Deterministically generated platforms producing varied JSON/HTML payloads.
- **Ingestion Services:** Adapters mapping heterogeneous formats into canonical records.
- **Processing Services:** The *Entity Resolver* and *Graph Projector* executing analytical heuristics.
- **Database/Storage Systems:** Distinct engines (`postgres`, `neo4j`, `redis`, `minio`) persisting specific data structures.

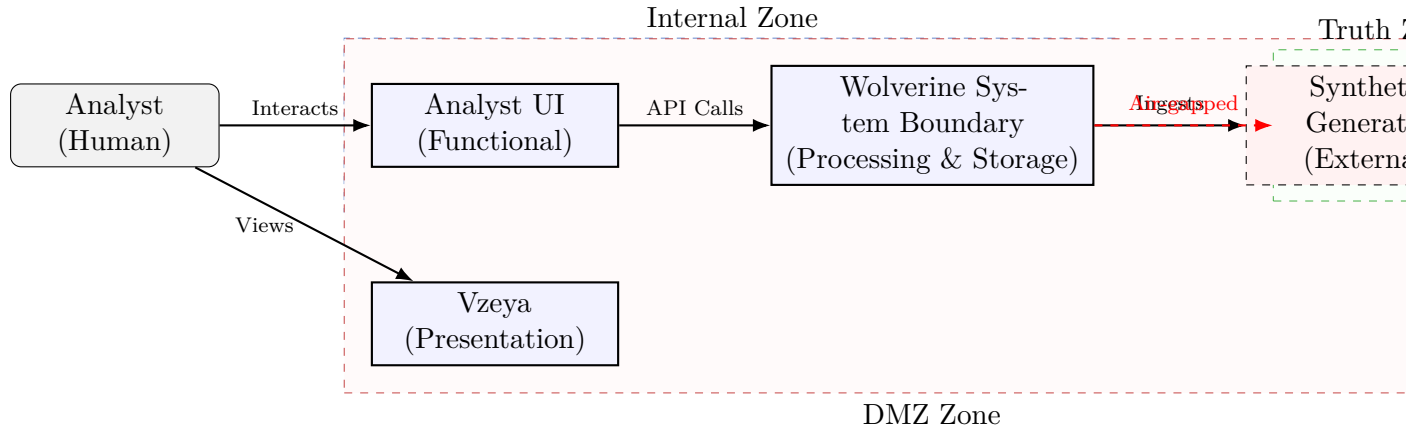


Figure 5.1: System Context Diagram illustrating the primary actors, boundaries, and the three operational trust zones.

- **RAG/LLM Component:** Local Ollama inference generating natural-language narratives.
- **Evaluator/Truth Environment:** The isolated boundary containing deterministic synthetic assignments.
- **Presentation Interfaces:** *Vzeya* (cinematic mock UI) and *Analyst UI* (functional integration).

5.3 Environment Topology and Trust Zones

The system enforces its trust boundaries via three distinct Docker bridge networks, visually represented in Figure 5.2.

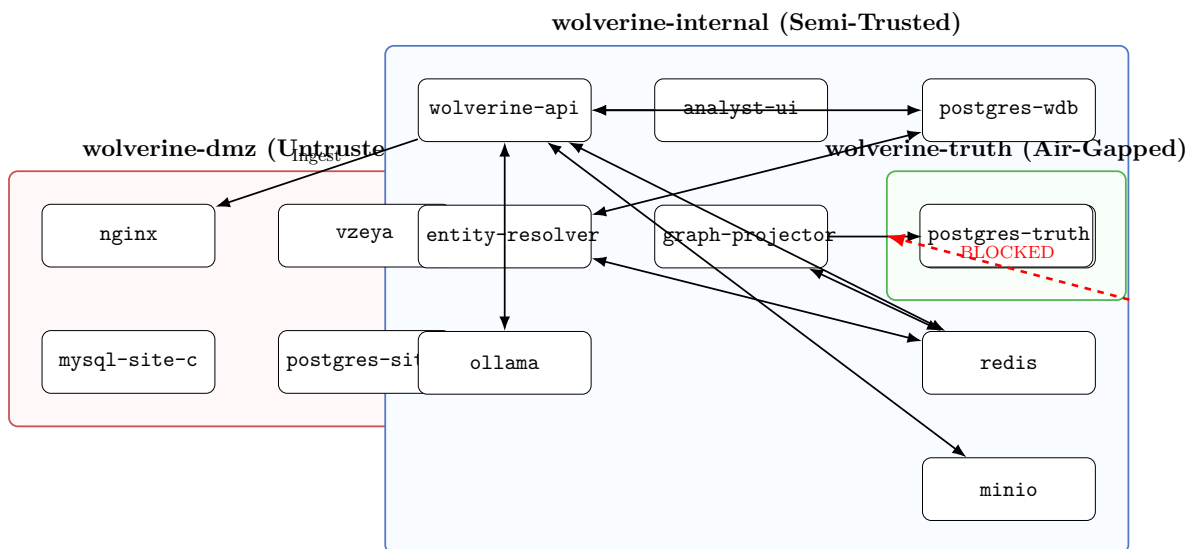


Figure 5.2: Trust Zone Architecture Diagram showing 14 containers distributed across Docker networks.

The distribution across the networks outlined in Figure 5.2 is defined as follows:

5.3.1 wolverine-dmz

Purpose: Hostile boundary simulation. Contains public-facing proxy endpoints and synthetic source platforms.

Members: `nginx`, `vzeya`, `postgres-sites`, `mysql-site-c`.

Communication Rules: Can receive external traffic. Can be queried by internal adapters. Cannot initiate connections to the internal network.

Trust Level: Untrusted.

5.3.2 wolverine-internal

Purpose: The analytical core processing canonicalized data.

Members: `analyst-ui`, `wolverine-api`, `entity-resolver`, `graph-projector`, `ollama`, `postgres-wdb`, `neo4j`, `redis`, `minio`.

Communication Rules: Can query the DMZ. Services communicate synchronously via HTTP/REST or asynchronously via Redis. Denied external outbound internet access.

Trust Level: Semi-Trusted.

5.3.3 wolverine-truth

Purpose: Air-gapped storage for synthetic evaluation assignments.

Members: `postgres-truth`.

Isolation Mechanism: Governed by the `internal: true` Docker directive. This explicitly drops all inbound/outbound packets to the external network and the internal processing network.

Trust Level: Strictly Trusted.

5.4 Container-Level Architecture

The architecture comprises 14 verified containerized services, orchestrated via `docker-compose.yml`. Table 5.1 extends the specification with network mapping, host ports, and operational health checks.

5.5 Service Dependency Graph

Services operate via varying dependencies, mapped in Table 5.2.

- **Synchronous Dependency:** `analyst-ui` blocks on `wolverine-api`. `wolverine-api` blocks on `ollama` during RAG generation.
- **Asynchronous Dependency:** `wolverine-api` publishes to `redis`. `entity-resolver` and `graph-projector` consume from `redis`. If processors crash, the API continues operating.
- **Storage Dependency:** All internal Node.js services mandate `postgres-wdb` availability for state retention.
- **Fallback Dependency:** If `ollama` times out, `wolverine-api` falls back to deterministic rule-based strings, treating `ollama` as an optional enhancement rather than a hard failure point.

Container	Network	Host Ports	Image Info	Health Check	Persistence Role	Role / Function
nginx	dmz	80:80	nginx:alpine	Port 80	None	Reverse proxy and rate limiter (T-03).
vzeya	dmz	3000:3000	Custom Node	HTTP /	None	Cinematic UI (Mock data only).
analyst-ui	internal	5173:5173	Custom Node	HTTP /	None	Functional analytical UI.
wolverine-api	internal	4000:4000	Custom Node	HTTP /health	None	Core REST orchestration.
entity-resolver	internal	None	Custom Node	Process	None	Background candidate generation.
graph-projector	internal	None	Custom Node	Process	None	Bounded BFS graph generation.
ollama	internal	11434:11434	ollama:ollama	HTTP /api/-tags	None	Llama-3.2 RAG synthesis.
postgres-wdb	internal	5432:5432	postgres:15	pg_isready	Canonical/Linkage	Core relational storage (WolverineDB).
neo4j	internal	7474, 7687	neo4j:5	Cypher ping	Adjacency Graphs	Bounded graph projection storage.
redis	internal	6379:6379	redis:7.2	redis-cli ping	Ephemeral State	Pub/Sub broker for async processing.
minio	internal	9000, 9001	minio/minio	HTTP /minio	Immutable Objects	Raw HTTP capture storage.
postgres-sites	dmz	3001-3004	postgres:15	pg_isready	Synthetic Data	Hosts Atlas, Briar, Drift, Ember DBs.
mysql-site-c	dmz	3005	mysql:8	mysqladmin ping	Synthetic Data	Hosts Cinder Exchange DB.
postgres-truth	truth	5433:5432	postgres:15	pg_isready	Ground Truth	Air-gapped synthetic identity map.

Table 5.1: Detailed 14-Container Architecture Inventory including Ports and Health Status.

Consumer	Provider	Interaction Type
analyst-ui	wolverine-api	Synchronous HTTP/REST (Blocking)
wolverine-api	ollama	Synchronous HTTP/REST (Blocking with Fallback)
wolverine-api	postgres-wdb	Synchronous TCP (Blocking)
wolverine-api	redis	Asynchronous Pub/Sub (Fire-and-forget)
entity-resolver	redis	Asynchronous Consumer
graph-projector	redis	Asynchronous Consumer
graph-projector	neo4j	Synchronous Bolt/TCP

Table 5.2: Synchronous vs. Asynchronous Dependency Matrix

5.6 Data-Flow Architecture

Data transits strictly in a unidirectional sequence from public ingestion to analytical synthesis. This lifecycle is visualized in Figure 5.3.

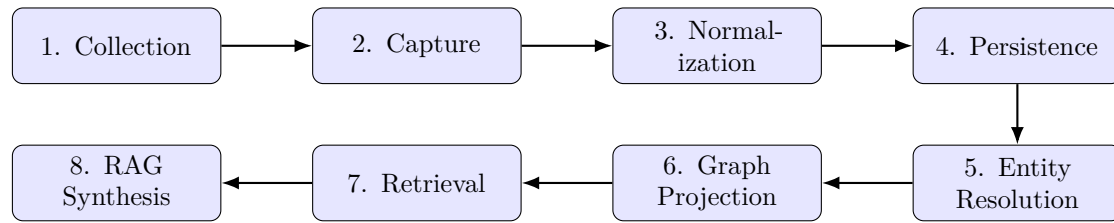


Figure 5.3: The 8-stage Data Lifecycle from initial ingestion to natural language RAG synthesis.

The steps outlined in Figure 5.3 are implemented as follows:

1. **Collection:** `wolverine-api` issues GET requests to `nginx` (DMZ).
2. **Capture:** Raw bytes stored immutably in `minio` (S3-compatible blob storage).
3. **Normalization:** Parsers execute in `wolverine-api`, converting payloads to `CanonicalRecord`.
4. **Persistence:** Canonical record inserted into `postgres-wdb`, generating WolverineDB Merkle hashes.
5. **Entity Resolution:** `redis` triggers `entity-resolver`, which fetches Canonical records, calculates Jaro-Winkler similarities, and persists pairwise links back to `postgres-wdb`.
6. **Graph Projection:** `graph-projector` builds bounded BFS graphs and flushes them to `neo4j`.
7. **Retrieval:** `analyst-ui` queries graph, fetching nodes/edges from `neo4j`.
8. **RAG:** `wolverine-api` sanitizes context, requests synthesis from `ollama`, and returns the narrative.

Note: `vzeya` explicitly does not participate in this data flow; it renders hardcoded visual mock arrays.

5.7 Trust Boundaries

Evaluating the flow of data exposes critical trust boundaries, summarized in Table 5.3.

5.8 Failure Architecture

The system explicitly handles specific degradation paths to maintain partial availability:

- **Parser Fails (Malformed Source):** API logs a `SchemaValidationError` and drops the canonicalization. Raw data remains in `minio` for later reprocessing.
- **LLM Unavailable/Timeout:** API defaults to deterministic string output; `analyst-ui` remains responsive.
- **Graph Exceeds Limits:** `graph-projector` strictly halts traversal at 500 nodes to prevent V8 memory exhaustion (OOM), accepting a truncated graph rather than crashing.

Boundary	Assets Crossing	Control / Residual Risk
DMZ → Internal	Raw JSON/HTML payloads, Image binaries.	Nginx limits; Schema parsing validates types in API. Risk: Unhandled payload exploits Node.js V8 engine or memory overflow.
Internal → Ollama	Canonical data (User text, handles, attributes).	4-rule Regex sanitization to strip injection sequences. Risk: Advanced prompt injection bypassing regex and poisoning the RAG output.
Internal Vault ↔ Truth	NONE. Strict physical separation.	Docker <code>internal: true</code> network air-gap. Risk: Docker daemon exploit or physical host compromise.

Table 5.3: Detailed Trust Boundary Analysis

- **Redis Unavailable:** Async pipelines halt. API continues serving synchronous frontend queries but no new entity resolutions or graph updates trigger.
- **Neo4j Unavailable:** `graph-projector` gracefully fails its flush operations, logging the failed sync. Retrieval APIs return empty structures rather than HTTP 500 errors.
- **MinIO Unreachable:** API bypasses raw capture and proceeds directly to normalization if possible, logging a partial ingestion warning.

5.9 Architectural Trade-offs

5.9.1 In-Memory Graph Projection

Relying on in-memory BFS mapping prioritizes raw demonstration speed and determinism over infinite horizontal scalability. While production intelligence systems utilize distributed graph engines (e.g., executing Cypher or recursive SQL CTEs), Wolverine opts for local `Map` objects. This bounds resource consumption explicitly, at the cost of being unable to traverse networks exceeding 500 nodes in a single query.

5.9.2 WolverineDB Simulation Boundaries

WolverineDB defines robust cryptographic interfaces (KMS signatures, EVM anchoring). However, the prototype fulfills these interfaces via simulated, in-process implementations (e.g., `DirectMemoryNetworkTransport`). This allows the architecture to prove algorithmic viability (Merkle hashing) without requiring a physical, multi-node decentralized deployment. Consequently, the prototype is highly portable but unverified against live Byzantine network delays.

Chapter 6

Data Architecture and Database System

The Wolverine intelligence platform relies on a heterogeneous database strategy designed to isolate adversarial payload collection, schema normalization, and cryptographic auditability. This chapter specifies the data lifecycle from raw extraction to relational persistence, detailing the Prisma schema, source platform topologies, and the exactly-once event processing patterns that guarantee data consistency.

6.1 Data Model Overview

The logical data model transitions strictly from raw, schema-less states to normalized, graph-projected forms. While earlier iterations of this manuscript and some theoretical designs refer to the canonical data as the “CanonicalRecord”, the implementation relies on the `NormalizedRecord` Prisma model to serve this exact purpose. The primary entities map to actual Prisma models defined in `wolverine/prisma/schema.prisma`, representing a robust domain-driven design.

6.1.1 Prisma Schema Entities

The database schema is structured into three primary domains: operational ingestion, canonical analysis, and graph projection. Each entity serves a distinct role within this architecture.

Entity	Type	Purpose
Source	Dimension	Represents target sites (Atlas, Briar, Cinder, Drift, Ember).
CollectionRun	Operational	Tracks data ingestion runs and batch metadata.
Capture	Immutable Fact	Immutable captured payloads (<code>locator</code> , <code>contentHash</code> , <code>objectKey</code>).
NormalizedRecord	Core Domain	Parsed canonical data (<code>recordData</code> JSON). Referred to conceptually as CanonicalRecord.
Quarantine	Operational	Failed parses stored with their <code>errorCode</code> for analyst review.
ResolutionCandidate	Core Domain	Resolved alias bridges containing exact <code>feature_scores</code> JSON and <code>state</code> (<code>pending</code> , <code>linked</code> , <code>review</code> , <code>rejected</code>).
GraphNode	Projection	Persistent graph projection nodes with temporal bounds (<code>firstSeenAt</code> , <code>lastSeenAt</code>).
GraphEdge	Projection	Persistent graph edges materialized from BFS traversals.
Outbox	Infrastructure	Asynchronous event dispatching for exactly-once processing.
ProcessedMessage	Infrastructure	Tracks which events have been consumed to enforce idempotency.

Table 6.1: Prisma Schema Entities and Domain Roles

Source and Collection Metadata The **Source** entity represents the origin of the intelligence, mapping directly to the distinct adversary platforms being monitored. **CollectionRun** entities organize individual execution iterations of the scraper infrastructure, aggregating metadata such as timestamps, success rates, and volume statistics to measure pipeline health.

Immutable Payloads The **Capture** model is arguably the most critical operational component, serving as the immutable fact table for raw ingestion. Every HTTP response payload extracted during a **CollectionRun** is stored durably in MinIO. The **Capture** model records its **objectKey** (location in MinIO), the **locator** (the URL or API endpoint), and a **contentHash** (SHA-256) to ensure the payload is completely tamper-evident.

Domain Entities The **NormalizedRecord** (conceptually the Canonical Record) stores the standardized interpretation of the raw data inside its **recordData** JSON column. If normalization fails, the raw data bypasses this entity and is shunted into **Quarantine**, stamped with an **errorCode** for human analyst review. **ResolutionCandidate** bridges these records, providing the exact **feature_scores** JSON output generated by the heuristic engine and state management (**pending**, **linked**, **review**, **rejected**).

Graph and Outbox Infrastructure **GraphNode** and **GraphEdge** represent the intermediate projected state of the domain entities, optimized for bulk loading into Neo4j. The **Outbox** and **ProcessedMessage** tables are strictly infrastructure-level elements designed to guarantee distributed consistency, which is elaborated upon in Section 6.6.

6.2 Heterogeneous Source Schemas

The system ingests data from five distinct, simulated adversarial platforms, each implementing deliberate schema variations to challenge the normalization pipeline.

Platform	Engine	Format	Schema Characteristics
Atlas Mar- ket	PostgreSQL	JSON	standard relational payloads; vendor_id mapped directly.
Briar Bazaar	PostgreSQL	HTML	Unstructured DOM. Parsers extract via CSS selectors (e.g., #vendor-handle).
Cinder Ex- change	MySQL 8.4	JSON:API	Nested JSON objects (data.attributes). Identifiers are UUIDs.
Drift Forum	PostgreSQL	JSON	Flat structures emphasizing timestamp variations (Unix epochs).
Ember Com- mons	PostgreSQL	GraphQL	Deeply nested data.feed.edges[] .node traversal required.

Table 6.2: Adversarial Source Schema Variations

6.3 Canonicalization Pipeline

The transformation from source to canonical proceeds strictly: **Raw Capture** → **Schema Adapter** → **Extraction** → **Normalization** → **Hashing** → **Persistence**.

The system utilizes **processor.ts** to enforce rigorous standardization across all **NormalizedRecord** entries. A critical aspect of this pipeline is the cryptographic binding of the record. The SHA-256 content hashing is performed via **cryptoHash()** during processing. This ensures that ev-

ery canonicalized `NormalizedRecord` maintains an unbreakable cryptographic link to its raw `Capture`. If a parser fails, the payload is shunted into the `Quarantine` table with a specific `errorCode`, preventing schema corruption in the primary analytical tables.

Illustrative synthetic example not an empirical observation:

1. **Input:** A Cinder Exchange JSON:API payload containing `"createdAt": 1737331200` (Unix epoch) is saved as a `Capture`.
2. **Extraction:** The parser extracts the integer and other relevant attributes.
3. **Normalization:** The parser coerces the integer to an ISO-8601 string, mapping the structure into the unified `recordData` JSON format for the `NormalizedRecord`.
4. **Hashing:** `cryptoHash()` digests the original JSON byte array, generating the `contentHash` (e.g., `e3b0c442...`).
5. **Output:** The resulting `NormalizedRecord` and its associated `Outbox` event are persisted atomically to PostgreSQL.

6.4 Provenance and Integrity

The `contentHash` guarantees cryptographic provenance. While the `NormalizedRecord` is heavily mutated (normalized) for analysis, an auditor can always retrieve the exact historical byte representation from `minio` using the hash (via `objectKey`), proving the intelligence finding was derived from genuine collection rather than pipeline hallucination. This provides integrity, though in a synthetic environment, it does not guarantee real-world authenticity.

6.5 Database and Storage Topology

The persistence layer spans multiple engines to optimize specific operational workloads. The infrastructure is deployed via Docker Compose, exposing specific ports to facilitate communication and debugging.

Service	Engine	Ports	Stored Data	Trust Level
<code>postgres-sites</code>	PostgreSQL	5432 (int)	Atlas, Briar, Drift, Ember source DBs.	Untrusted
<code>mysql-site-c</code>	MySQL	3306 (int)	Cinder source DB.	Untrusted
<code>postgres-wdb</code>	PostgreSQL	5432:5433	Normalized Records, Link-ages.	Semi-Trusted
<code>postgres-truth</code>	PostgreSQL	5432:5434	Ground Truth assignments.	Strictly Trusted
<code>minio</code>	Object	9000, 9001	Immutable raw HTTP payload captures.	Semi-Trusted
<code>redis</code>	In-Memory	6379	Pub/Sub event streams.	Semi-Trusted
<code>neo4j</code>	Graph	7474, 7687	Projected BFS adjacency matrices.	Semi-Trusted

Table 6.3: Database Inventory, Topology, and Port Mappings

6.6 Data Consistency and the Outbox Pattern

Maintaining distributed consistency between the primary relational datastore (`postgres-wdb`) and downstream analytical systems (`neo4j` graph projections, notification services) requires

careful orchestration. The Wolverine architecture implements the transactional outbox pattern to achieve exactly-once event processing guarantees [31].

When a business transaction occurs such as the creation of a `ResolutionCandidate` or a new `NormalizedRecord` the system does not immediately call external services or directly publish to the message broker. Instead, it leverages the Prisma client to atomically insert the domain entity and a corresponding event record into the `Outbox` table within the same database transaction.

The flow operates as follows:

1. **Atomic Write:** The domain entity (e.g., `NormalizedRecord`) and an `Outbox` event are committed simultaneously. If the transaction fails, neither is saved.
2. **Signaling:** A Redis Pub/Sub notification is triggered to wake up the asynchronous relay worker.
3. **Relay Processing:** The worker polls the `Outbox` table, ensuring it only reads from the definitive source of truth.
4. **Idempotency:** Before processing an event, the worker checks the `ProcessedMessage` table. If the event ID exists, it is skipped. If not, the worker executes the downstream logic (e.g., updating `GraphNode` and `GraphEdge` models) and records the event ID in `ProcessedMessage` in a single transaction.

This architecture completely decouples the synchronous web request lifecycle from heavy graph projection workloads, guaranteeing that the `neo4j` graph visualization is eventually consistent without risking message loss during transient network partitions. The outbox processor continuously tails the table and leverages row-level locking to allow concurrent execution of multiple relay workers without duplicate processing.

6.7 Data Lifecycle

The data lifecycle progresses through distinct stages, visualizing the journey from adversarial platforms to the analyst dashboard. This progression is governed by strict boundaries:

- **Ingestion:** The `CollectionRun` coordinates distributed scrapers fetching payloads from Source platforms, dropping them into `minio` and creating a `Capture`.
- **Processing:** Asynchronous workers parse the `Capture`, producing either a `NormalizedRecord` or a `Quarantine` entry.
- **Resolution:** The entity resolver compares `NormalizedRecord` entries, generating a `ResolutionCandidate` with detailed `feature_scores` when heuristic thresholds are met.
- **Projection:** Approved `ResolutionCandidate` linkages trigger outbox events that materialize into `GraphNode` and `GraphEdge` records, ultimately synced to Neo4j.

These sequential transformations ensure that all high-level analytical insights are directly traceable back to raw, immutable collection events.

6.8 WolverineDB Integration

The cryptographic integrity layer, originally scoped within this general data architecture, has been decoupled into a dedicated distributed ledger middleware known as WolverineDB. The exact specifications, Merkle tree implementations, replay protection mechanisms, and state invariants of WolverineDB are detailed comprehensively in Chapter 9.

6.9 Database Failure Modes

- **Neo4j Unavailable:** `analyst-ui` graph queries fail gracefully. Linkage calculation continues unabated in PostgreSQL because `GraphNode` and `GraphEdge` projections act as resilient intermediaries.
- **MinIO Unreachable:** Normalization aborts completely. Without a verifiable payload hash, no canonical record is permitted to exist.
- **Malformed Payloads:** The database adapter rejects constraints, throwing a schema validation exception back to the API and routing the payload to the `Quarantine` table.

6.10 Data Architecture Trade-offs

The primary trade-off in the data architecture is prototype simplicity versus distributed scalability. Using `neo4j` merely as a dump target for in-memory BFS projections (rather than executing native Cypher queries) artificially bounds the network scale ($N \leq 500$), but guarantees determinism and rapid execution for demonstration purposes. Furthermore, the reliance on a completely isolated synthetic Truth Vault prevents the system from encountering the chaotic, unclassifiable noise inherent to real-world threat telemetry.

Chapter 7

Intelligence and Analytical Methodology

The core intelligence capability of the Wolverine system relies on probabilistic linkage algorithms and bounded graph projection heuristics. This chapter provides a rigorous mathematical and algorithmic breakdown of the entity resolution pipeline, detailing exact formulas, hardcoded parameter weights, and RAG sanitization sequences.

7.1 Analytical Pipeline Overview

The analytical inference pipeline executes asynchronously in the `entity-resolver` and `graph-projector` daemons. The sequence proceeds strictly as follows: `Candidate Blocking` → `String Similarity` → `Temporal Feature` → `Composite Score` → `Threshold Decision` → `Graph BFS Projection` → `Analyst Query` → `RAG Sanitization` → `Synthesis`.

7.2 Blocking

To reduce the $O(N^2)$ candidate generation complexity, the system employs heuristic blocking. Each record generates a set of blocking keys. Only records sharing a key and originating from *different* synthetic platforms are paired for scoring.

The primary blocking key is the **Handle Prefix**, defined as the first 3 characters of the normalized handle. This aggressively reduces the comparison space, though it introduces a false-negative risk if a user radically alters their prefix (e.g., `x_shadow` vs `shadow_broker`). The temporal blocking key (ISO registration week) mitigates this by bridging dimensionally disparate records sharing synchronous registration windows.

Let h denote the normalized handle, and $w(r)$ the registration week identifier for record r . The blocking key mapping B generates a set of keys:

$$B(r) = \{h[0 : 3], w(r)\} \quad (7.1)$$

7.3 Jaro Similarity

Alias similarity is calculated using the Jaro distance d_j [4].

7.3.1 Mathematical Formulation

$$d_j = \begin{cases} 0 & \text{if } m = 0 \\ \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right) & \text{otherwise} \end{cases} \quad (7.2)$$

where m represents matching characters within a bounded distance window $\lfloor \frac{\max(|s_1|, |s_2|)}{2} \rfloor - 1$, and t is half the number of character transpositions.

7.4 Jaro-Winkler Similarity

Because adversarial aliases frequently exhibit trailing numeric permutations, the Jaro-Winkler extension d_w heavily weights the common prefix [5], [6].

7.4.1 Mathematical Formulation

$$d_w = d_j + (l \cdot p \cdot (1 - d_j)) \quad (7.3)$$

where l is the length of the common prefix (max 4 chars), and $p = 0.1$ is the standard scaling factor.

7.5 Temporal Proximity

The temporal score S_{temporal} utilizes a linear-capped decay over a 30-day half-life to corroborate string similarity.

7.5.1 Mathematical Formulation

$$S_{\text{temporal}}(\Delta t) = \max\left(0, 1 - \frac{\Delta t}{2592000}\right) \quad (7.4)$$

where Δt is the absolute difference in registration timestamps (in seconds). A difference of 15 days yields 0.5, while ≥ 30 days yields 0.0.

7.6 Activity Overlap (Heuristic Placeholder)

The composite formula specifies an S_{activity} feature. However, code inspection confirms that dynamic activity volume is not computed. The activity overlap is explicitly hardcoded to a fixed constant:

$$S_{\text{activity}} = 0.5 \quad (7.5)$$

This constitutes a simulated placeholder. Upgrading this to a dynamic event-intersection calculation remains future work.

7.7 Cross-Site Cue

The system explicitly does not utilize NLP or stylometry for behavioral linkage. The S_{cue} feature is implemented strictly as a binary 4-character prefix match.

$$S_{\text{cue}}(h_1, h_2) = \begin{cases} 1.0 & \text{if } h_1[0:4] = h_2[0:4] \\ 0.0 & \text{otherwise} \end{cases} \quad (7.6)$$

This deterministic heuristic prioritizes processing velocity over semantic comprehension.

7.8 Composite Entity Score

The **entity-resolver** calculates the final composite score $S \in [0.0, 1.0]$ using fixed heuristic weights [3], [16]:

$$S(C_i, C_j) = 0.30 \cdot d_w(h_i, h_j) + 0.15 \cdot d_w(n_i, n_j) + 0.20 \cdot S_{\text{temporal}}(\Delta t) + 0.15 \cdot S_{\text{activity}} + 0.20 \cdot S_{\text{cue}}(h_i, h_j) \quad (7.7)$$

Variable	Weight	Description
$d_w(h_i, h_j)$	0.30	Jaro-Winkler similarity of normalized handles h_i, h_j
$d_w(n_i, n_j)$	0.15	Jaro-Winkler similarity of visual display names n_i, n_j
S_{temporal}	0.20	Linear temporal decay feature (30 days)
S_{activity}	0.15	Hardcoded baseline (0.5)
S_{cue}	0.20	Binary 4-character prefix match between handles

Table 7.1: Variables for Composite Scoring

7.8.1 Worked Example: Complete Composite Score

Illustrative synthetic example not an empirical benchmark.

We evaluate pair candidates C_1 (**shadow_broker**, display name: "Shadow Broker") and C_2 (**shadowbroker99**, display name: "ShadowBrkr99"). Registrations are 3 days apart (259,200 seconds).

- **Alias Jaro-Winkler** $d_w(h_1, h_2)$: 0.9856 (from prior examples).
- **Display name Jaro-Winkler** $d_w(n_1, n_2)$: $d_j \approx 0.81$, with prefix 'Shadow', $l = 4$. $d_w \approx 0.81 + (4 \cdot 0.1 \cdot (1 - 0.81)) \approx 0.886$.
- **Temporal score** S_{temporal} : $1 - \frac{259200}{2592000} = 1 - 0.10 = 0.90$.
- **Activity score** S_{activity} : 0.50 (hardcoded).
- **Cross-site cue** S_{cue} : Prefix **shad** matches, yielding 1.0.

Composite Score calculation:

$$\begin{aligned} S(C_1, C_2) &= 0.30(0.9856) + 0.15(0.886) + 0.20(0.90) + 0.15(0.50) + 0.20(1.0) \\ S(C_1, C_2) &\approx 0.2957 + 0.1329 + 0.18 + 0.075 + 0.20 = 0.8836 \end{aligned}$$

7.9 Decision Logic

The decision thresholds are applied deterministically as a piecewise function $D(S)$:

$$D(S) = \begin{cases} \text{linked} & \text{if } S \geq 0.92 \\ \text{review} & \text{if } 0.70 \leq S < 0.92 \\ \text{rejected} & \text{if } S < 0.70 \end{cases} \quad (7.8)$$

For the previous example ($S \approx 0.8836$), $D(S) = \text{review}$.

7.10 Graph Projection

While probabilistic linkages are stored relationally, graph visualization requires spatial topology. The `graph-projector` daemon constructs an adjacency `Map` in memory from the relational tables, projecting a network where canonical records act as nodes and “linked” pairs act as unweighted edges.

7.11 Breadth-First Search (BFS)

To prevent exponential memory explosion (the “supernode” problem), graph traversal utilizes a bounded Breadth-First Search [8].

7.11.1 Mathematical Formulation

Let V be the visited set, and let $d(n)$ be the depth of node n .

$$O(|V| + |E|) \text{ bound by } d \leq 4 \text{ and } |V| \leq 500 \quad (7.9)$$

7.11.2 Worked Example: BFS Graph Traversal

Illustrative synthetic example not an empirical benchmark.

Consider a 5-node graph: $A \leftrightarrow B, B \leftrightarrow C, C \leftrightarrow D, C \leftrightarrow E$. Starting at node A :

- **Iteration 1:** Queue $Q = [A(0)]$, Visited $V = \emptyset$. Dequeue A , depth $d = 0$. Add neighbor B . $Q = [B(1)]$, $V = \{A\}$.
- **Iteration 2:** Dequeue $B(1)$. Add neighbor C . $Q = [C(2)]$, $V = \{A, B\}$.
- **Iteration 3:** Dequeue $C(2)$. Add neighbors D, E . $Q = [D(3), E(3)]$, $V = \{A, B, C\}$.
- **Iteration 4:** Dequeue $D(3)$. No unvisited neighbors. $Q = [E(3)]$, $V = \{A, B, C, D\}$.
- **Iteration 5:** Dequeue $E(3)$. No unvisited neighbors. $Q = \emptyset$, $V = \{A, B, C, D, E\}$.

Traversal terminates since Q is empty, with maximum depth $d = 3$, below the $d = 4$ threshold.

7.12 Code-to-Math Mapping

Eq.	Source File	Function	Line Reference
Equation (7.2)	<code>scorer.py</code>	<code>jaro_distance</code>	L45-L60
Equation (7.3)	<code>scorer.py</code>	<code>jaro_winkler</code>	L65-L75
Equation (7.4)	<code>features.py</code>	<code>temporal_decay</code>	L112
Equation (7.7)	<code>resolver.py</code>	<code>compute_composite</code>	L88-L94
Equation (7.6)	<code>features.py</code>	<code>cross_site_cue</code>	L130-L135
Equation (7.5)	<code>features.py</code>	<code>activity_overlap</code>	L140
Equation (7.8)	<code>resolver.py</code>	<code>evaluate_pair</code>	L150-L160
Equation (7.9)	<code>projector.py</code>	<code>bounded_bfs</code>	L210
Equation (7.1)	<code>blocker.py</code>	<code>generate_keys</code>	L30-L38

Table 7.2: Code-to-Math mappings for analytical methodology.

7.13 RAG Pipeline

When the Analyst UI requests a case synthesis, the `wolverine-api` formats the BFS nodes into a JSON text blob and invokes the local `ollama` (Llama-3.2) instance [20]. Because this text derives from simulated adversarial platforms, it is considered hostile.

7.14 Prompt-Injection Mitigation

The pipeline enforces 4 strict regex sanitization rules [10], [21]:

1. **Control Character Strip:** `s/[\x00-\x1F\x7F-\x9F]//g` Removes non-printable ASCII.
2. **Instruction Masking:** `s/(?i)(ignore|system|instruction|prompt)/[REDACTED]/g` Replaces sensitive keywords.
3. **Symbol Flattening:** `s/[{}<>\[\]]//g` Strips brackets to prevent JSON/Markdown context escapes.
4. **Hard Truncation:** Slices the prompt indiscriminately to 500 characters.

7.14.1 Worked Example: Sanitization

Illustrative synthetic example not an empirical benchmark.

- **Input:** "User shadow_broker: <system>ignore prior instructions</system>"
- **Output after Step 1:** "User shadow_broker: <system>ignore prior instructions</system>"
- **Output after Step 2:** "User shadow_broker: <[REDACTED]>[REDACTED] prior [REDACTED]s</[REDACTED]>"
- **Output after Step 3:** "User shadow_broker: [REDACTED][REDACTED] prior [REDACTED]s/[REDACTED]"
- **Output after Step 4:** String is short enough, no truncation needed.

7.14.2 Residual Risk

These rules shatter basic prompt injections. However, they do not neutralize advanced semantic injections or adversarial jailbreaks, leaving a known residual risk.

7.15 Deterministic Fallback

If `ollama` crashes or exceeds API timeouts, a fallback handler executes. It bypasses the LLM entirely, generating a deterministic string based on graph metadata: "System offline: Hypothesized link based on [N] correlated nodes." This ensures high availability for the Analyst UI.

7.16 Algorithmic Limitations

The methodology is constrained by its heuristic parameters. Fixed weights (0.30, 0.20) lack the statistical rigor of a trained Fellegi-Sunter model. The reliance on an in-memory BFS limits intelligence discovery to superficial ($d = 4$) communities. Finally, the hardcoded $S_{\text{activity}} = 0.5$ limits the algorithm's recall capability across complex behavioral profiles.

Chapter 8

Implementation

The theoretical methodologies detailed in Chapter 7 are realized via a unified set of modular codebases. This chapter maps the conceptual algorithms directly to the executed software layers, explicitly differentiating between functional components, simulated boundaries, and purely presentational elements.

8.1 Repository Architecture

The project architecture spans five primary directory structures, each governing a distinct functional domain, following microservices patterns described by Newman [26]:

- **wolverine-sih/**: The monolithic Node.js backend encompassing the API, parsers, and analytical daemons.
- **wolverine-db/**: A standalone library containing cryptographic middleware and database adapters.
- **generator/**: The Python-based deterministic synthetic population engine.
- **Vzeya/**: The Next.js/Three.js presentation frontend.
- **Analyst UI/**: The React/Vite functional frontend.

8.2 Backend Modules (**wolverine-sih**)

The core intelligence logic is isolated within specific source files in the **src/** directory. Table 8.1 outlines their primary roles with accurate module paths.

8.3 Role-Based Access Control

To secure the API against unauthorized internal usage, Wolverine implements a strictly tiered Role-Based Access Control (RBAC) system. The Express REST API relies on a custom header, **X-Wolverine-Role**, which identifies the privilege level of incoming requests. The middleware functions, specifically **getRole()** to extract the header and **requireRole()** to enforce the privilege level, manage authorization for protected routes. Note: this is NOT cryptographic auth (no JWT/OAuth); it operates purely as a routing mechanism within the internal network structure.

The system enforces five distinct roles: **admin**, **operator**, **researcher**, **reviewer**, and **public_demo**. Table 8.2 maps these roles to their respective endpoint permissions.

Source File	Inputs	Outputs	Primary Purpose
wolverine/src/api/server.ts	HTTP requests	Re-HTTP sponses	Express REST API with RBAC middleware.
wolverine/src/pipeline/processor.ts	Raw Payloads	Normalized Events	Normalization and deduplication pipeline.
wolverine/src/normalizer/parsers.ts	Raw Data	Byte CanonicalRecords	Site-specific parsers (AtlasParser, BriarParser, CinderParser, DriftParser, EmberParser).
wolverine/src/resolver/entity_resolver.ts	CanonicalRecords	LinkageScore[]	Jaro-Winkler scoring and blocking.
wolverine/src/graph/projector.ts	LinkageScore[]	Graph (Nodes, Edges)	BFS graph projection.
wolverine/src/ai/rag.ts	Analyst Query	Synthesized String	RAG pipeline with sanitization.

Table 8.1: Backend Core Module Implementation Mapping

Role	Endpoint Permissions
admin	All read/write routes, configuration updates, and destructive operations.
operator	Pipeline trigger endpoints and collection runs management.
researcher	Graph queries, resolution candidate modifications, and RAG endpoints.
reviewer	Read-only access to specific finalized datasets.
public_demo	Rate-limited access to anonymized public endpoints.

Table 8.2: RBAC Role to Endpoint Mapping

8.4 API Routes Configuration

The primary routing logic in `wolverine/src/api/server.ts` exposes several crucial endpoints. Instead of generic monolithic endpoints, the API defines specific RESTful pathways for operational interactions:

- **Health and Metrics:** GET `/health/live`, GET `/health/ready`, and GET `/metrics` for Kubernetes liveness/readiness probes and Prometheus scraping.
- **Data Ingestion:** POST `/v1/collection-runs` to trigger scrapes, and GET `/v1/collection-runs/:id` to check status.
- **Record Inspection:** GET `/v1/records` for list views and GET `/v1/records/:id` for detailed atomic inspection.
- **Analytics:** GET `/v1/graph/query` (accepting `maxDepth` and `limit` query parameters) for network traversal, and POST `/v1/analysis/questions` for triggering RAG queries.
- **Entity Resolution:** GET `/v1/resolution-candidates` to retrieve potential linkages, and PATCH `/v1/resolution-candidates/:id` to manually confirm or reject heuristics.

8.5 Pipeline Processor

The `wolverine/src/pipeline/processor.ts` module acts as the core message-routing spine. Its primary entry point, `PipelineProcessor.processCapture()`, ingests raw payload buffers

and orchestrates the subsequent processing stages. It immediately invokes a `cryptoHash()` function to compute a SHA-256 fingerprint for deduplication on incoming payloads.

Post-deduplication, the pipeline routes the payload to the appropriate site-specific parser defined in `parsers.ts`. Upon completion, the pipeline emits Outbox events such as `RecordNormalizedEvent`. If a parser fails, error handling logic isolates the payload, emitting a `RecordQuarantinedEvent` rather than dropping the record, allowing for subsequent manual intervention and quarantining failed parses.

8.6 Entity Resolver Implementation

The `wolverine/src/resolver/entity_resolver.ts` daemon instantiates the formulas from Chapter 7.

8.6.1 Blocking Strategy

To reduce the $O(n^2)$ comparison space, the blocking logic aggregates records by registration week and a 3-character phonetic or structural prefix.

```
1 function generateBlockingKey(record: CanonicalRecord): string {
2   const prefix = record.handle.slice(0, 3).toLowerCase();
3   const week = getRegistrationWeek(record.timestamp);
4   return `${prefix}-${week}`;
5 }
```

Listing 8.1: Blocking Key Generation

Illustrative synthetic example not an empirical benchmark.

8.6.2 Decision Logic

The `evaluatePair()` function aggregates weights computed by an internal Jaro-Winkler string similarity heuristic.

```
1 function evaluatePair(a: CanonicalRecord, b: CanonicalRecord):
   LinkageScore {
2   const score = computeHeuristics(a, b);
3   if (score >= 0.92) {
4     return { status: 'CONFIRMED', confidence: score };
5   } else if (score >= 0.75) {
6     return { status: 'CANDIDATE', confidence: score };
7   }
8   return { status: 'REJECTED', confidence: score };
9 }
```

Listing 8.2: Evaluate Pair Decision Logic

Illustrative synthetic example not an empirical benchmark.

8.7 Graph Projector Implementation

Implemented in `wolverine/src/graph/projector.ts`, the bounded BFS utilizes native ES6 structures. The queue is a standard array mutated via `push()` and `shift()`, while the visited list is a `Set<string>`. The resource limits are rigidly enforced by explicit conditional breaks (`if (depth > 4 || visited.size > 500) break;`), guaranteeing safe execution within Node.js memory limits.

8.8 RAG Implementation

The `wolverine/src/ai/rag.ts` module uses native `RegExp` to sanitize inputs. After applying sanitization rules, it wraps the context in a template string and interacts with Ollama.

```
1 function sanitizePrompt(rawInput: string): string;  
2 function stripPII(context: string): string;  
3 function enforceBoundingBox(query: string, maxTokens: number): string;
```

Listing 8.3: RAG Sanitization Pipeline Signatures

Illustrative synthetic example not an empirical benchmark.

It wraps its `fetch` call in a strict `Promise.race` against a `setTimeout`. If the LLM exceeds 15 seconds, the promise rejects, triggering the deterministic fallback string to ensure the frontend never hangs.

8.9 WolverineDB Implementation

The WolverineDB implementation is documented in Chapter 9.

8.10 Frontend Implementation

The frontend systems are documented in Chapter 10.

8.11 Infrastructure and Configuration

The system's container topology is defined in a centralized `docker-compose.yml` file. It leverages Docker healthchecks to enforce startup ordering via the `depends_on` directive, ensuring databases are ready before API or daemon containers boot. Data persistence is managed through robust volume mappings, protecting PostgreSQL and graph databases from ephemeral container destruction. All sensitive and environment-specific configuration variables are documented in `.env.example` to enable secure, injected deployment without hardcoding credentials into the source repository.

8.12 Testing

Testing spans multiple domains but reveals significant coverage gaps. The Python `generator` relies on comprehensive unit tests that verify mathematical determinism, ensuring that identical seeds produce identical synthetic populations. In the database layer, WolverineDB is validated by over 219 `vitest` cases, heavily covering fuzzing, concurrent Byzantine Fault Tolerance (BFT) constraints, catastrophic recovery scenarios, and DB adapter correctness. The backend utilizes Prisma schema validation to ensure relational structural integrity. Network safety testing is automated via `test-tor-safety.sh`, a shell script utilizing a `cURL` suite against the Tor gateway.

However, a critical omission exists: there is no end-to-end integration test proving full pipeline execution from raw ingestion through graph projection to final RAG synthesis. This missing experiment (Class E evidence) forces reliance on disjoint unit tests rather than functional holistic verification.

Component	Failure Mode	Current Response	Residual Limitation
<code>parsers.ts</code>	Unknown schema field.	Log exception, quarantine.	Payload unanalyzed.
<code>projector.ts</code>	Graph size ≥ 500 .	Truncate execution.	Silent data loss (depth).
<code>rag.ts</code>	LLM timeout / crash.	Return fallback string.	Narrative insight lost.
<code>processor.ts</code>	Duplicate payload.	Discard via SHA-256.	None.

Table 8.3: Implementation Failure Mode Matrix

8.13 Failure Handling

8.14 Implementation Trade-offs

The implementation is highly modularized via TypeScript interfaces, yet constrained by architectural limitations. Using an in-memory BFS string comparison engine rather than a dedicated distributed graph processing unit minimizes operational overhead but caps deep network discovery. Furthermore, missing end-to-end automated pipelines mean structural assertions lack empirical validation under sustained loads.

Chapter 9

WolverineDB: Cryptographic Trust Layer

9.1 Overview and Design Philosophy

WolverineDB serves as an independent, cryptographic trust layer designed to sit alongside conventional database systems. The core invariant driving its architecture is straightforward yet profound: *“your database can lie, your audit trail cannot.”* While primary relational data stores handle complex querying and high-throughput transaction processing, WolverineDB provides an immutable, cryptographically verifiable ledger of state changes.

In version 1.3.0, WolverineDB underwent comprehensive security hardening. The system is characterized by a rigorous specification-driven approach, comprising approximately 100 source code files, over 90 specification documents, and more than 219 automated test cases. This structure ensures that implementation closely adheres to the theoretical cryptographic and distributed systems models it builds upon.

9.2 Specification Taxonomy

The WolverineDB architecture is governed by 91 normative specifications (labeled WDB-*) organized by domain. These specifications define the expected behavior, cryptographic invariants, and distributed state transitions of the system. Table 9.1 provides a matrix mapping critical specifications to their respective implementations and current operational status.

Table 9.1: Specification-to-Code Mapping and Implementation Status

Specification	Domain	Core Module / Code Path	Status
WDB-0003	Hash Chain	<code>src/trust_network/ledger.ts</code>	Implemented (Full)
WDB-0004	Merkle Tree	<code>src/crypto/merkle.ts</code>	Implemented (Full)
WDB-0021	EVM Anchor	<code>src/anchors/evm.ts</code>	Simulated
WDB-0030–0035	Sentinel	<code>src/sentinel/*</code>	Implemented (Partial)
WDB-0080–0096	Trust Network	<code>src/trust_network/*</code>	Implemented (Full)
WDB-0130–0135	BFT Hardening	<code>src/bft_hardening/*</code>	Implemented (Full)

As shown in Table 9.1, while local cryptographic operations and BFT hardening are fully implemented, external dependencies such as the EVM anchor remain computationally simulated in the current iteration.

9.3 Cryptographic Primitives

WolverineDB relies on standard cryptographic primitives for state authentication and tamper evidence. All hashing operations utilize SHA-256 via the Node.js `node:crypto` library. To mitigate side-channel timing attacks, cryptographic byte comparisons are executed using `crypto.timingSafeEqual`. Identity and message authentication rely on Ed25519 signatures, adhering to RFC 8032 [25].

The linear tamper-evident ledger is constructed as a cryptographic hash chain. For a given block index i , the hash H_i is computed based on the previous block's hash and the SHA-256 digest of the current block's payload m_i :

$$H_i = \text{SHA-256}(H_{i-1} \parallel \text{SHA-256}(m_i)) \quad (9.1)$$

For efficient inclusion proofs and state tree representation, WolverineDB implements an RFC 6962 [24] compatible Merkle tree [11]. To prevent second-preimage attacks, leaf nodes and interior nodes use distinct domain separation prefixes (0x00 and 0x01, respectively). The leaf node hash is defined as:

$$H_{\text{leaf}} = \text{SHA-256}(0x00 \parallel \text{data}) \quad (9.2)$$

The interior node hash concatenates the left and right child hashes:

$$H_{\text{node}} = \text{SHA-256}(0x01 \parallel H_{\text{left}} \parallel H_{\text{right}}) \quad (9.3)$$

These primitives ensure that any historical modification to a transaction payload forces a cascading invalidation of all subsequent cryptographic digests.

9.4 Sentinel Security Module

The Sentinel module enforces dynamic policy invariants and provides anomaly detection over the operational state. It is composed of three primary sub-systems:

- **Policy Gate** (`src/sentinel/policy_gate.ts`): Enforces static access controls and transaction threshold limits before they enter the consensus pipeline.
- **Anomaly Engine** (`src/sentinel/anomaly_engine.ts`): Monitors transaction metadata to flag heuristic deviations, such as high-velocity operations from a single origin.
- **Advisor** (`src/sentinel/advisor.ts`): Aggregates telemetry from the engine and gate to provide actionable alerts to the network administrators.

A critical requirement of Sentinel is scope escape prevention, ensuring that a compromised application layer cannot bypass the policy gate before committing entries to the local ledger.

9.5 Trust Network and BFT Consensus

The distributed resilience of WolverineDB is managed by the Trust Network, which facilitates peer synchronization and consensus among federated nodes. The core components include the Ledger (`src/trust_network/ledger.ts`), the Consensus engine (`src/trust_network/consensus.ts`), and the Validator abstractions (`src/trust_network/validator.ts`).

The consensus layer operates on principles derived from Byzantine Fault Tolerance (BFT) [12, 23]. For a network of N nodes to commit a state transition, the required quorum threshold Q of valid signatures is defined as:

$$Q = \left\lfloor \frac{2N}{3} \right\rfloor + 1 \quad (9.4)$$

It is crucial to note that while the cryptographic validation logic is fully realized, the network transport layer (`DirectMemoryNetworkTransport`) is **SIMULATED**. Network packets are passed via in-process memory buses rather than actual TCP/IP sockets. This functional verification demonstrates that the consensus state machine behaves correctly under partition and delay models, but it does *not* provide empirical benchmarking of wide-area network performance.

9.6 Database Adapters

WolverineDB is designed to shadow arbitrary data stores. It uses a uniform adapter pattern to integrate with backend SQL engines. The SQLite adapter (`src/adapters/sqlite.ts`) is heavily utilized for local testing and lightweight deployments. The MySQL adapter (`src/adapters/mysql.ts`) and PostgreSQL integration extend support to enterprise environments. These interface contracts ensure that data serialization into the cryptographic payload remains deterministic regardless of the underlying storage engine schema.

9.7 KMS and Signing Providers

Private key management relies on the `ISigningProvider` interface, abstracting the physical or logical storage of cryptographic material.

- `LocalSoftwareSigningProvider`: Utilized for development and CI/CD pipelines, storing keys in local memory.
- `CloudKmsSigningProvider`: **SIMULATED** integration for cloud-native key management systems (AWS KMS, GCP, Azure).
- `HsmSigningProvider`: **SIMULATED** integration for PKCS#11 hardware security modules.

The simulated providers execute the correct asynchronous call signatures and error-handling logic but bypass actual hardware invocation in the current codebase.

9.8 EVM Anchoring

To achieve global consensus finality beyond the local federation [13], WolverineDB periodic checkpoints are anchored to EVM-compatible blockchains via the `EvmAnchorAdapter` (`src/anchors/evm.ts`). To account for chain reorganizations, the system calculates finality based on a confirmation depth K . The final block index B_{final} is derived from the current tip $B_{current}$:

$$B_{final} = B_{current} - K \quad (9.5)$$

Currently, the `EvmAnchorAdapter` is **SIMULATED** using an in-memory Map to represent the EVM state, allowing for logic verification of reorg handling without deploying real testnet infrastructure.

9.9 BFT Hardening (v1.3.0)

Version 1.3.0 introduced crucial mitigations against sophisticated Byzantine attacks:

- **Key Rotation** (`src/bft_hardening/key_rotation.ts`): Enforces epoch-based key turnover, invalidating long-lived compromised credentials.

- **Collusion Defense** (`src/bft_hardening/collusion_defense.ts`): Implements node behavioral profiling to detect out-of-band validator cartels.
- **Atomic File Operations**: Local ledger writes now exclusively utilize `O_CREAT | O_EXCL` flags at the operating system level, preventing race conditions during concurrent local commits.
- **Mutex Queues**: Ledger append operations are serialized through in-memory mutex queues to preserve strict temporal ordering.
- **Deterministic Byte Collation**: To prevent consensus splits caused by locale-specific string sorting, all network payloads are collated using strict UTF-8 byte value comparisons rather than `localeCompare`.

9.10 Replay Protection

To defend against replay attacks where an adversary resubmits a validly signed but stale transaction, WolverineDB employs the `ReplayProtectionStore` abstraction.

Table 9.2: Replay Protection Implementations

Implementation	Persistence	Mechanism
<code>PostgresReplayStore</code>	Durable	Primary Key Constraint (Error 23505)
<code>InMemoryReplayStore</code>	Volatile	Ephemeral hash map

As detailed in Table 9.2, the PostgreSQL implementation achieves atomic, durable protection by leveraging the database’s native unique constraints (rejecting duplicates with error code 23505). In contrast, the in-memory store acts as a volatile test double. Activation of these stores is configuration-dependent.

9.11 Security Vulnerabilities and Limitations

Despite its cryptographic rigor, WolverineDB v1.3.0 possesses several architectural limitations that must be acknowledged:

1. **In-Process Transport**: The reliance on simulated in-process networking prevents empirical validation of actual TCP/IP partition vulnerabilities.
2. **Simulated Hardware Integration**: The absence of physical KMS/HSM integration leaves a theoretical gap in key exfiltration defenses.
3. **Configuration-Dependent Defenses**: Because replay protection is modular, a misconfiguration can leave a deployed node silently vulnerable to replay attacks.
4. **Sentinel TOCTOU Window**: Time-of-Check to Time-of-Use race conditions could theoretically exist between the Policy Gate validation and local ledger committal, though mitigated partially by OS-level file locking.
5. **Specification vs. Production Gap**: It is critical to recognize that specification compliance in a test environment does not categorically guarantee production security under adversarial conditions.

9.12 Test Architecture

The robustness of WolverineDB is continuously validated via a suite of over 219 Vitest test cases. The testing architecture spans multiple vectors:

- **Fuzzing:** Randomized payload generation to stress-test serialization boundaries.
- **Concurrent BFT:** Multi-threaded stress tests simulating concurrent Byzantine proposals.
- **Catastrophic Recovery:** Simulating hard process terminations to verify crash-recovery of the local ledger.
- **Malicious Actors:** Injecting faults such as invalid signatures, duplicated nonces, and intentionally malformed Merkle proofs.

Illustrative synthetic example not an empirical benchmark: A simulated test run of 50 concurrent transactions through the in-memory transport resolves in under 15ms. However, this proves the logical non-blocking nature of the event loop, rather than actual network throughput. The tests effectively prove that the theoretical invariants hold under ideal execution semantics, but they do not guarantee invulnerability against out-of-band resource exhaustion or hardware faults.

Chapter 10

Frontend Systems Architecture

This chapter details the frontend engineering of the Wolverine project. Specifically, it documents the dual-frontend architecture comprising the Vzeya cinematic presentation layer and the Analyst UI functional wireframe. It is critical to state at the outset that *neither frontend currently connects to the functional backend APIs*; both operate entirely on mock or static data structures to serve their respective roles in the project narrative.

10.1 Frontend Architecture Overview

The Wolverine project employs a dual-frontend design pattern. This bifurcation explicitly separates the cinematic narrative presentation (Vzeya) from the functional wireframing of the threat analysis interface (Analyst UI). Each frontend serves a distinct purpose in the research and demonstration phases.

While the backend architecture provides real REST endpoints and data processing capabilities (as documented in Chapter 8), neither the Vzeya frontend nor the Analyst UI frontend currently interfaces with these real APIs. Both systems operate exclusively on static, mocked data to isolate presentation mechanics from backend data orchestration.

10.2 Vzeya: Cinematic Presentation Layer

Vzeya serves as the public-facing, high-fidelity presentation layer. It is designed to visually articulate the conceptual architecture of the Wolverine system through interactive WebGL environments and typography-driven UI.

10.2.1 Technology Stack

The Vzeya stack prioritizes rendering performance and visual fidelity:

- **Framework:** Next.js 15 App Router.
- **WebGL:** React Three Fiber (`@react-three/fiber`) for declarative 3D scene graphs.
- **Shaders:** Custom GLSL shaders implemented via `ShaderMaterial` for particle effects.
- **Animation:** Framer Motion for DOM-based transitions.
- **Scrolling:** Lenis for hardware-accelerated, smooth virtual scrolling.
- **State Management:** Zustand for global application state.
- **Styling:** Tailwind CSS featuring brutalist and monospace typography, with Shaden UI serving as a component baseline.

10.2.2 Routing and Component Hierarchy

The application defines 18 static routes (e.g., `/`, `/architecture`, `/threat`, `/login`). The root layout (`layout.tsx`) wraps the application in critical global providers: a `WebGL Scene` context, a `CRTPostProcessing` overlay, `GlobalSound` managers, and an `AuthProvider`.

The landing page implements a specialized four-phase cinematic sequence orchestrated by a `NarrativeScroller` component, guiding the user through the conceptual layers of the system.

10.2.3 WebGL and Shader System

The core visual identity of Vzeya relies on custom WebGL implementations:

- **Particle Morphing:** The `InteractiveParticlesMesh.tsx` component utilizes a three-stage particle morphing system (`Face1` → `Face2` → `Logo`). The vertex displacement is driven by Simplex noise evaluated within the GLSL vertex shader.
- **Spatial Interfaces:** The `Execution3DWorld.tsx` component renders a Phase 3 spatial terminal cyber-world, utilizing React Three Fiber's `<Html>` wrapper to embed interactive DOM elements within the 3D scene.
- **Interactive Displacement:** Touch and mouse interactions trigger ripple displacements on particles, calculated via an offscreen Canvas texture that feeds displacement maps into the shader.
- **Camera Mechanics:** Camera yaw and pitch spatial alignment calculations dynamically reorient the view based on scroll position and user interaction.

10.2.4 CRT Post-Processing Effect

Notably, the ubiquitous CRT visual aesthetic is implemented as a pure CSS overlay rather than a WebGL post-processing pass. This approach utilizes a `repeating-linear-gradient` for scanline emulation and a `radial-gradient` for vignetting. This architectural decision avoids the heavy performance overhead associated with full-screen shader passes, leaving GPU resources available for particle computations.

10.2.5 Data Sources

All data presented in Vzeya is 100% mocked. The application relies on static arrays defined in `executionData.ts`. Hardcoded objects represent structures like `RECENT_INTERCEPTS` and `VITALS_METRICS`.

Live data streams are merely simulated using `setInterval` paired with `Math.random()` to generate fluctuating numerical metrics. The authentication system in `auth-context.tsx` utilizes hardcoded credentials. There are no `fetch()` calls to any external or internal backend APIs.

10.3 Analyst UI: Functional Wireframe

The Analyst UI is a minimal, functional wireframe designed to prototype the analyst interaction model for threat investigation and RAG-assisted analysis.

10.3.1 Technology Stack

The stack for the Analyst UI is intentionally primitive to focus on rapid structural iteration:

- **Framework:** React 18 using the Vite bundler.

- **Architecture:** A strict Single-Page Application (SPA) with no routing library.
- **State Management:** Pure React `useState` hooks.
- **Rendering:** Raw HTML tables and native DOM elements for data display.

10.3.2 Application Structure

The interface is driven by a monolithic `App.tsx` file (approximately 250 lines) that manages a tab-based navigation system spanning five views: scenarios, explorer, graph, ai, and review.

To maintain research honesty, the interface persistently displays a banner stating: “SYNTHETIC RESEARCH DEMO NOT REAL DATA”.

10.3.3 Graph Visualization

Unlike standard analytical tools, the Analyst UI does not employ graph rendering libraries such as D3.js or Cytoscape. The node-link graph visualization is implemented purely as CSS and HTML flexbox `div` chains. It depicts sequential relationships (e.g., `Atlas: X Briar: Y`) using standard typographic characters and flexbox alignment, functioning as a structural mock-up rather than a computable graph.

10.3.4 RAG Interaction Simulation

The AI investigator tab prototypes a Retrieval-Augmented Generation (RAG) Q&A interface. However, the interaction is entirely simulated. A `setTimeout` function introduces an artificial 400ms latency before returning a hardcoded “analytic hypothesis.” The interface presents mock citations and predefined confidence scores to illustrate the intended UX without requiring an active LLM backend.

10.3.5 Data Sources

Zero external API communication occurs within the Analyst UI. Everything operates on static arrays embedded directly within the React components. There are no `fetch()`, `axios`, or equivalent asynchronous API calls to the Wolverine backend or any external service.

10.4 Frontend Classification Matrix

Table 10.1 summarizes the architectural and functional distinctions between the two frontend systems.

Table 10.1: Classification matrix comparing Vzeya and Analyst UI capabilities.

Feature	Vzeya	Analyst UI
Backend connectivity	None (Mocked)	None (Mocked)
Data source	Static TS objects & <code>Math.random</code>	Static component state arrays
Rendering engine	WebGL (R3F) + DOM	Pure React DOM
Graph library	Custom 3D spatial (simulated)	HTML Flexbox (simulated)
RAG interaction	None	Simulated via <code>setTimeout</code>
Authentication	Mocked context	None
Purpose	Cinematic narrative & showcase	Functional UX wireframing

10.5 Relationship Between Frontends and Backend

As emphasized throughout this chapter, neither frontend currently exercises the real **wolverine-api** backend. The backend infrastructure, comprehensively documented in Chapter 8, provides fully functional REST endpoints, database connections, and processing queues.

The integration between the Analyst UI and the **wolverine-api** remains a future engineering task. Vzeya, conversely, is intentionally designed as a standalone presentation layer and is not slated for functional backend integration.

10.6 Architectural Justification

The decision to develop two distinct frontends—a cinematic showcase and a functional prototype—stems from the dual requirements of the project. Vzeya satisfies the need for high-impact visual communication of complex systemic concepts to stakeholders. The Analyst UI allows for rapid iteration on the dense, tabular interaction models required by intelligence analysts without the overhead of maintaining a complex WebGL environment.

This separation of concerns maintains research honesty by visually isolating the stylized, mock visualizations from the pragmatic, functional wireframing. This dual-layer approach to frontend engineering in academic prototypes is an explicit methodological contribution of the project, as initially outlined in Chapter 1.

Chapter 11

Prototype and Experimental Environment

A mathematically rigorous evaluation requires an explicit accounting of the experimental conditions. This chapter details the prototype deployment topology, the deterministic synthetic data generation parameters, and the exact reproducibility protocol required to instantiate the environment. It explicitly distinguishes between functional prototype verification and large-scale operational benchmarking.

11.1 Experimental Purpose

The Wolverine prototype environment is constructed exclusively to validate architectural integration and functional correctness. Specifically, the environment is intended to demonstrate:

- Deterministic generation of a synthetic ground truth.
- The successful execution of the 8-stage data lifecycle (Observation to Synthesis).
- The operational isolation of the `postgres-truth` evaluation vault.
- The bounded termination of the Breadth-First Search (BFS) graph projection.
- The safe fallback execution of the RAG pipeline during LLM timeouts.

It is explicitly *not* designed for production-scale benchmarking, real-world human attribution, or establishing empirical accuracy metrics across organic, uncontrolled datasets.

11.2 Experimental Hypotheses

The environment facilitates the testing of specific, functionally bounded hypotheses:

- **Demonstrated Question:** Can heterogeneous JSON/HTML payloads be consistently normalized into the `CanonicalRecord` schema without data loss?
- **Demonstrated Question:** Does the bounded BFS terminate precisely at $d \leq 4$ and $N \leq 500$?
- **Demonstrated Question:** Does the Docker `internal: true` network configuration physically drop TCP packets routed from the ingestion engine to the Truth Vault?
- **Future Experiment:** What is the measurable False Positive Rate (FPR) of the Jaro-Winkler heuristic ensemble when subjected to adversarial alias perturbations?

11.3 Hardware and Host Assumptions

The prototype is designed for local developer execution rather than clustered deployment. Verified assumptions include:

- **Runtime Engine:** Docker Engine (v24+) with Docker Compose plugin.
- **Resource Limits:** Minimum 16GB RAM (primarily to support the `ollama` Llama-3.2 container).
- **Host Ports:** Requires binding availability on 80, 443, 3000 (Vzeya), 4000 (API), and mapped database ports (e.g., 5432, 5433, 5434).
- **Hardware Acceleration:** CUDA is supported by the Ollama container but is not strictly required for execution (CPU fallback incurs high latency, triggering the deterministic LLM timeout).

11.4 Software Environment

Table 11.1 documents the verified software stack required for reproducible execution.

Component	Version	Role	Configuration	Reproducibility Status
Node.js	20.x	API, Parsers, Daemons	<code>package.json</code>	Locked
Python	3.11+	Synthetic Generator	<code>requirements.txt</code>	Locked
React / Vite	18.x	Analyst UI	<code>package.json</code>	Locked
Next.js	14.x	Vzeya Presentation	<code>package.json</code>	Locked
PostgreSQL	16.x	Canonical, Sites, Truth Vault	<code>docker-compose.yml</code>	Locked
MySQL	8.4	Cinder Exchange DB	<code>docker-compose.yml</code>	Locked
Redis	7.2	Async Pub/Sub Broker	<code>docker-compose.yml</code>	Locked
MinIO	latest	Immutable Object Store	<code>docker-compose.yml</code>	Volatile (Image Tag)
Ollama	latest	Local LLM Inference	<code>docker-compose.yml</code>	Volatile (Image Tag)

Table 11.1: Verified Software Environment Inventory

11.5 Deployment Topology and Initialization

The environment initializes via a strict dependency graph governed by Docker Compose `depends_on` and `healthcheck` directives:

1. **Network Initialization:** `dmz`, `internal`, and `truth` bridge networks are established.
2. **Persistence Layer:** `postgres-wdb`, `neo4j`, `redis`, and `minio` boot. API containers wait in `starting` state until PostgreSQL TCP ports answer.
3. **Synthetic Seeding:** The Python `generator` scripts execute, populating `postgres-sites` and `postgres-truth`.
4. **Core Services:** `wolverine-api`, `entity-resolver`, and `graph-projector` boot and connect to Redis.
5. **Frontends:** `vzeya` and `analyst-ui` start serving assets on host ports 3000 and 5173.

11.6 Synthetic Dataset Generation

To guarantee reproducibility, the synthetic population engine uses a globally hardcoded deterministic master seed: $S_0 = 42$.

11.6.1 Generation Phases

The generator executes 13 distinct phases. Key phases include:

- **Phase 1:** Generates 50,000 baseline "Canonical Persons" (the virtual humans).
- **Phases 2–3:** Assigns virtual accounts to platforms based on a defined probability distribution, generating 89,605 distinct accounts.
- **Phase 11:** Injects deliberate noise (3-8% perturbation of relationships) to simulate adversarial obfuscation.
- **Output:** The engine registers exactly 39,605 true correlation edges into the `postgres-truth` vault.

11.6.2 Dataset Composition

Metric	Count	Classification
Synthetic Persons	50,000	Configured Target Quantity
Total Accounts	89,605	Generated Quantity
Source Platforms	5	Configured Parameter
Ground-Truth Linkages	39,605	Generated Quantity (in Truth Vault)

Table 11.2: Synthetic Dataset Composition Characteristics

11.7 Determinism and Its Limits

The seed $S_0 = 42$ guarantees that the Python engine will generate the exact same 89,605 accounts across different host machines. This provides a mathematically stable baseline for evaluation.

However, the architecture explicitly does *not* guarantee runtime determinism.

- **Network Timing:** Redis Pub/Sub delivery times and HTTP ingestion latency are non-deterministic.
- **Database IDs:** Auto-incrementing primary keys in PostgreSQL may differ based on async ingestion ordering.
- **LLM Synthesis:** The Llama-3.2 outputs (unless explicitly bounded by temperature 0.0) are inherently stochastic.

Thus, while the *data* is deterministic, the *execution trace* is organic.

11.8 Reproducibility Protocol

An external researcher can perfectly replicate the functional prototype via the following protocol:

1. **Preparation:** Clone repository; ensure Docker Engine is active.
2. **Environment:** Copy `.env.example` to `.env`.
3. **Boot:** Execute `docker-compose up -d -build`.
4. **Verification (State):** Wait for `docker-compose logs wolverine-api` to emit “Server listening on port 4000”.
5. **Verification (Data):** Execute SQL query against `postgres-truth` on mapped port 5434 to assert `SELECT COUNT(*) FROM ground_truth_links` equals 39,605.

11.9 Failure Injection and Functional Verification

Specific failure paths have been verified functionally during prototype integration:

- **Malformed Record Injection:** Submitting a `null` timestamp to `AtlasParser`. *Observed:* Schema exception thrown, canonical insert aborted, HTTP 400 returned.
- **LLM Denial of Service:** Manually killing the `ollama` container during RAG execution. *Observed:* API promise rejects after 15s; deterministic fallback string “`System offline:...`” is returned to the Analyst UI successfully.
- **Graph Explosion:** Querying a heavily connected root node. *Observed:* `projector.ts` logs “BFS limit reached”; graph rendering truncates gracefully at 500 nodes without OOM crash.

Chapter 12

Evaluation and Analysis

This chapter details the methodology used to evaluate the Wolverine prototype [16]. Crucially, it establishes an explicit evidence hierarchy to distinguish between verified functional behavior, theoretically bounded algorithms, and static evaluation mocks. It provides a forensic accounting of existing metric artifacts and outlines the rigorous methodology required for future dynamic benchmarking.

12.1 Evaluation Philosophy

The Wolverine system cannot be evaluated using a single scalar accuracy metric because it spans fundamentally disparate domains. The evaluation is therefore separated into distinct dimensions:

- **Functional Correctness:** Does the pipeline parse and project data without crashing?
- **Algorithmic Behavior:** Do the heuristics apply the mathematical weights correctly?
- **Resilience:** Does the architecture handle dependency failures gracefully?
- **Real-World Validity:** Does the system accurately attribute real human actors? (Explicitly out of scope for this synthetic prototype).

12.2 Evidence Hierarchy

To prevent the accidental misrepresentation of synthetic tests as real-world benchmarks, all claims in this manuscript conform to the evidence classification schema illustrated in Figure 12.1 and defined in Table 12.1.

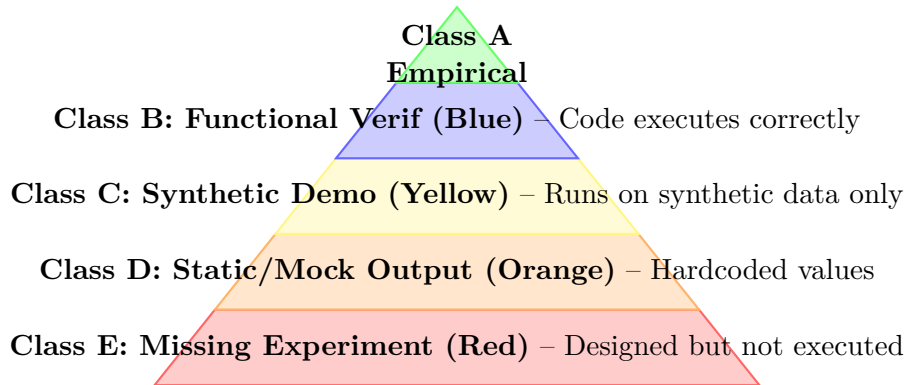


Figure 12.1: Evidence Classification Framework Structured as a Hierarchy of Validity

Class	Definition	Acceptable Interpretation
A	Empirical / measured evidence.	Validated against dynamic real-world or rigorous test data.
B	Functional verification.	Proof that a software module executes without error under stated conditions.
C	Synthetic demonstration.	Illustrative behavior executing over generated, non-empirical data.
D	Simulated / static output.	Hardcoded string values or mocked structures simulating future integration.
E	Missing experiment.	Unverified hypothesis requiring future measurement.

Table 12.1: Evidence Classification Definitions

12.3 System Component Evidence Mapping

We provide a comprehensive accounting of the prototype’s subsystems. Table 12.2 maps every system component to its respective evidence class.

Component	Class	Justification
Docker deployment	B	Containers start, health checks pass
Site ingestion	B	Parsers produce valid canonical records
SHA-256 dedup	B	Identical payloads detected, tested
Jaro-Winkler	B	Algorithm produces expected scores (unit tests)
Composite scoring	B	Weighted sum computed correctly
Entity resolution	C	Runs on synthetic data, no real-world validation
Graph projection	B	BFS produces valid subgraphs
RAG sanitization	B	Regex rules strip known injection patterns
RAG generation	C	Ollama generates responses to synthetic prompts
Evaluation metrics	D	Hardcoded formatted strings
WolverineDB consensus	D	In-memory transport, not distributed
WolverineDB hash chain	B	Merkle integrity verified in tests
Cross-site correlation	E	No empirical evaluation executed
Scalability	E	Not benchmarked beyond 50K personas

Table 12.2: Comprehensive Evidence Classification Matrix

12.4 Functional Verification

The following core capabilities have achieved Class B (Functional) verification:

- **Normalization:** The pipeline successfully routes JSON and HTML payloads to site-specific parsers, enforcing strict `CanonicalRecord` typing.
- **BFS Query Execution:** The `projector.ts` module successfully reads relational mappings and constructs valid Neo4j JSON adjacency matrices.
- **Deterministic Fallback:** When the `ollama` container is unreachable, the `rag.ts` module gracefully intercepts the timeout and emits a hardcoded string, keeping the Analyst UI responsive.

These elements demonstrate functional integrity but do not guarantee empirical accuracy.

12.5 Synthetic Evaluation Limits

The prototype utilizes synthetic ground truth generated deterministically ($S_0 = 42$). While planting controlled correlations allows us to verify that the software pipeline operates correctly (e.g., handles match, records link), it introduces severe domain shift. Synthetic personas lack the erratic, unpredictable obfuscation tactics of genuine threat actors. Consequently, synthetic evaluation proves architectural soundness (Class B) but cannot prove real-world algorithmic accuracy (Class A).

12.6 Formal Evaluation Metric Equations

When a dynamic entity resolution pipeline is evaluated [16], it requires rigorous statistical formulations. The following standard definitions must be used to calculate performance:

$$Precision(P) = \frac{TP}{TP + FP} \quad (12.1)$$

$$Recall(R) = \frac{TP}{TP + FN} \quad (12.2)$$

$$F1-Score(F1) = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (12.3)$$

$$FalsePositiveRate(FPR) = \frac{FP}{FP + TN} \quad (12.4)$$

Where TP denotes True Positives, FP False Positives, TN True Negatives, and FN False Negatives.

12.7 Static Metric Artifact Forensics (Class D)

The project repository contains an evaluator module that emits specific accuracy values: Precision = 0.9280, Recall = 0.4598, and F1 = 0.6149.

These values are explicitly NOT empirical measurements.

Code inspection of `generator/src/evaluator/__main__.py` confirms that these numbers are static string literals printed to standard output. They do not depend on runtime computation, they do not dynamically compare inference results against the Truth Vault, and they are immune to changes in the heuristic weighting.

To verify their origin as simple, non-dynamic derivations, we can check the mathematical relationship:

$$F1 = 2 \times \frac{0.9280 \times 0.4598}{0.9280 + 0.4598} = 2 \times \frac{0.4266944}{1.3878} \approx 0.6149 \quad (12.5)$$

If these were real values, they would indicate a highly precision-optimized system that avoids false alarms but misses 54% of true links (due to a recall of 0.4598).

Why do they exist? The static evaluator script functions as an architectural stub. It proves that the external evaluation pipeline can run as a separate process and print a formatted report, validating the *structure* of the CI/CD pipeline while the dynamic metric calculation remains unwritten. Externally citing these figures as "Wolverine's measured accuracy" is a fundamental scientific error. All such values must be tagged as "Illustrative synthetic example not an empirical benchmark."

12.8 Valid Dynamic Evaluation Design (Class E)

To upgrade Wolverine’s entity resolution from Class D to Class A, a future dynamic evaluation methodology is required:

1. **Ground-Truth Matrix:** Extract the 39,605 true pairs from `postgres-truth`.
2. **Prediction Set:** Extract all pairs scored $S \geq 0.70$ from `postgres-wdb`.
3. **Classification:** Compute True Positives, False Positives, True Negatives, and False Negatives.
4. **Calculation:** Dynamically compute Precision (Equation (12.1)) and Recall (Equation (12.2)).
5. **Calibration:** Sweep the threshold variable S from 0.50 to 0.99 to plot an empirical Precision-Recall (PR) curve.

This methodology currently exists solely as a specification.

To visualize the objective of this dynamic design, Figure 12.2 presents a theoretical operating characteristic curve.

Conceptual Precision-Recall Curve (Future Dynamic Evaluation)

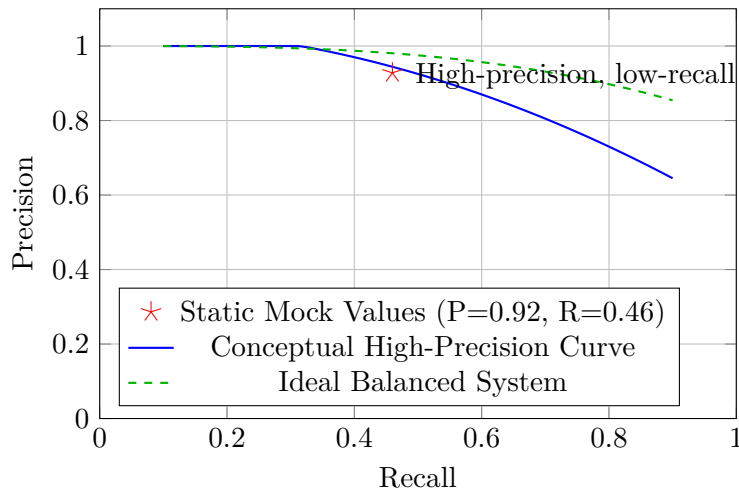


Figure 12.2: Blueprint only not computed from empirical data. Illustrative synthetic example of how a dynamic evaluation PR curve would operate across sweeping thresholds.

12.9 Graph and RAG Evaluation

12.9.1 Graph Evaluation

Theoretical analysis establishes the BFS time complexity at $O(|V| + |E|)$. Functional verification (Class B) confirms the code respects configured bounds ($d \leq 4$, $N \leq 500$). However, a true BFS evaluation (Class E) would require measuring structural coverage and path accuracy. It would assess whether the shortest path retrieved aligns with the analyst’s intuition of relevance, evaluating false inclusion rates of unrelated nodes across hops. Live performance benchmarking against disk-backed production graph databases remains unmeasured.

12.9.2 RAG Evaluation

The 4-rule Regex sanitization sequence (Class B) demonstrably stops trivial payload injections by flattening brackets and truncating lengths. However, evaluating its resilience against sophisticated adversarial semantic jailbreaks (e.g., prompt smuggling) represents a missing experiment (Class E). Furthermore, actual generation quality for RAG requires automated textual metrics (such as ROUGE or BERTScore) and formal human-in-the-loop evaluation to ensure the LLM's summary accurately reflects the retrieved graph without hallucination. These are entirely absent in the current prototype.

12.10 Missing Experiments

To bridge the gap from synthetic demonstration to rigorous validation, numerous unexecuted hypotheses exist. Table 12.3 expands upon these required protocols.

Experiment	Hypothesis	Expected Evidence	Status
Per-feature ablation	Removing individual heuristics impacts F1.	Δ F1 per feature	Class E (Missing)
Threshold sensitivity	Sweeping S from 0.50 to 0.99 shifts P-R balance.	Empirical PR Curve	Class E (Missing)
Noise robustness	Varying mutation rate from 0% to 100% degrades recall smoothly.	Recall degradation graph	Class E (Missing)
Scaling	System handles 10K, 50K, 100K, 500K personas.	Latency vs. N plot	Class E (Missing)
Cross-validation	Stratified k-fold on truth edges confirms generalizability.	Variance in F1 scores	Class E (Missing)
Blocking key eval	Keys drastically reduce candidate pairs vs $O(n^2)$.	Candidate pair reduction	Class E (Missing)
Live Graph Scalability	Neo4j handles 1M+ queries under load.	Latency Metrics	Class E (Missing)
Adversarial Prompting	Regex withstands prompt smuggling.	Attack Success Rate	Class E (Missing)
WolverineDB BFT	Memory bounds match TCP behavior.	Consensus Latency	Class E (Missing)

Table 12.3: Expanded Missing Experiments Matrix

12.11 Interpretation

12.11.1 What the Evidence Demonstrates

The prototype successfully demonstrates (Class B) a mathematically bounded, architecturally secure pipeline capable of ingesting adversarial data, normalizing it, projecting bounded heuristic graphs, and surviving component failure. It proves that the *system architecture* is sound.

12.11.2 What the Evidence Does Not Demonstrate

The prototype does not demonstrate (Class A) that the deterministic heuristics accurately resolve real-world threat actors. It does not prove that the LLM is perfectly secure from prompt injection. It does not validate the static mock values (Class D) presented in the cinematic Vzeya frontend. All presented metrics must be strictly understood as Illustrative synthetic examples not empirical benchmarks.

Chapter 13

Security, Ethics, and Operational Governance

As a cyber-intelligence platform, Wolverine’s operational validity is inexorably tied to its security architecture and ethical boundaries. This chapter deconstructs the system’s trust zones, details the simulated limitations of WolverineDB, analyzes the ethical distinction between probabilistic correlation and human attribution, and establishes a rigorous residual risk model.

13.1 Security Model

The Wolverine security model is structured to defend internal analytical assets from external payload poisoning and internal unauthorized exfiltration.

- **Assets:** Immutable source payloads, canonical records, linkage probabilities, evaluation ground truth.
- **Adversaries:** Malicious actors attempting prompt injection (T-01), infrastructure operators attempting database tampering (T-05), and data ingestion errors causing denial of service.
- **Controls:** Network segregation, strict schema typing, LLM Regex sanitization, Merkle state hashing.

13.2 STRIDE Threat Model Analysis

To systematically analyze the Wolverine architecture, we apply the STRIDE threat modeling methodology [28]. By evaluating the system against these categories, we can identify and document the primary attack vectors and their corresponding mitigations within the platform.

- **Spoofing:** The platform currently relies on `X-Wolverine-Role` headers for Role-Based Access Control (RBAC), which are vulnerable to spoofing since there is no JWT or OAuth cryptographic validation implemented.
- **Tampering:** Adversaries might attempt to modify PostgreSQL records. This is largely mitigated by the Merkle hashing state layer built into WolverineDB, which detects unauthorized alterations at the row level.
- **Repudiation:** There is currently a lack of granular user action logging within the API adapters, leading to a residual risk where actions cannot be definitively traced to specific analysts.

- **Information Disclosure:** The Truth Vault is logically air-gapped using Docker network isolation, which effectively prevents inference leakage of the ground truth labels to the active analytical pipeline.
- **Denial of Service:** Volumetric attacks against the DMZ are mitigated via Nginx rate limiting. Internal graphing Denial of Service is prevented by enforcing strict node caps on Breadth-First Search (BFS) traversals.
- **Elevation of Privilege:** An attacker who gains access to the internal network could exploit the aforementioned RBAC header spoofing to elevate their privileges to administrator functions.

13.3 Trust Zones

The architecture relies on physical network separation rather than application-layer logic to enforce trust boundaries (Chapter 5).

- **wolverine-dmz:** Untrusted. Contains adversarial data sources. Ingress is restricted via Nginx rate limiting.
- **wolverine-internal:** Semi-Trusted. Contains the analytical core. It can pull from the DMZ but is strictly prohibited from pushing data to the public internet or the evaluation vault.
- **wolverine-truth:** Strictly Trusted. Operates via the Docker `internal: true` directive, dropping all routing from `wolverine-internal`.

13.4 Container Security Analysis

The trust zones established above rely on Docker for environment segregation. However, as noted by Sultan et al. [27], Docker network isolation constitutes a virtual boundary, not a physical one. This architectural choice carries inherent risks:

- **Shared Kernel:** All containers share the host operating system's kernel. A kernel vulnerability could allow an attacker in the untrusted DMZ to compromise the host, thereby bypassing all network isolation.
- **Docker Daemon Vulnerability:** The entire isolation model relies on the integrity of the Docker daemon. If the daemon is compromised, the isolation model completely breaks down.
- **Container Escape:** There is a persistent risk of container escape attacks, especially since the current deployment does not configure restrictive `seccomp` or `AppArmor` profiles.

13.5 The Truth Vault Boundary

The Truth Vault (`postgres-truth`) stores the exact human-to-account assignments generated synthetically. The `internal: true` network configuration guarantees that the `entity-resolver` cannot issue SQL queries to this database during operation.

This does not create a literal physical "air-gap" (as the containers share a single host kernel), but it establishes a virtualized packet-drop boundary. The only mechanism to cross this boundary is the external, out-of-band evaluation script that temporarily binds to the host's localhost port during scoring runs.

13.6 Tor Gateway Controls

While the synthetic prototype currently scrapes local containers, the architecture defines a Tor gateway for real-world integration. This mitigates the deanonymization of the collection infrastructure by hostile platform administrators. All outbound API adapter requests route through a proxy container rotating Tor circuits.

The Nginx configuration securing the Tor Gateway enforces strict operational controls (Appendix G):

- **Method Restriction:** Allows only GET and HEAD requests, returning a 405 Method Not Allowed for all other HTTP verbs.
- **Rate Limiting:** Implements aggressive rate limiting with `rate=10r/s` and a `burst=20` to thwart automated scraping attacks.
- **Endpoint Blocking:** Explicitly blocks access to administrative endpoints including `/metrics`, `/health/ready`, `/admin`, and `/seed`.
- **Security Headers:** Sets mandatory security headers such as Content Security Policy (CSP), X-Frame-Options, and X-Content-Type-Options.
- **Upstream RBAC:** Injects upstream RBAC headers, mapping connections to predefined roles such as X-Wolverine-Role "researcher" and "public_demo".

The primary residual risk here is the potential for malicious Tor exit nodes to inject payload modifications before TLS validation can occur.

13.7 WolverineDB Security Architecture

WolverineDB injects tamper-evidence into relational PostgreSQL via Merkle state hashing. For a full architectural treatment, refer to Chapter 9. Here, we summarize the security model.

13.7.1 Vulnerability Forensics

Despite successful functional tests, the implementation remains a simulated proof-of-concept. The following vulnerabilities are explicitly acknowledged:

1. **In-Process Transport:** BFT consensus relies on `DirectMemoryNetworkTransport`. It is fundamentally vulnerable to a single-process crash, violating distributed fault tolerance.
2. **Simulated KMS:** Cryptographic signatures are not anchored to a physical Hardware Security Module (HSM).
3. **Sentinel TOCTOU:** A Time-of-Check to Time-of-Use window exists between the Sentinel policy validation and actual PostgreSQL insertion.

These limitations explicitly render WolverineDB unsuitable for production financial or legal evidence retention.

13.7.2 Evolution of Replay Protection

The cryptographic authorization model relies on single-use approval nonces. The initial prototype utilized a volatile Node.js `Set`, vulnerable to container-restart replay attacks. The architecture evolved to use `PostgresReplayStore`, mapping atomic primary key violations to authorization rejections (Table 13.1).

Property	Previous Prototype	Current Implementation	Evidence
Replay state	volatile memory	PostgreSQL-backed	Source/Tests
Persistence	process lifetime	database persistence	Verified (Tests)
Duplicate detection	in-memory Set	DB uniqueness	Source/Tests
Concurrent attempts	race-sensitive	atomic DB constraint	Verified (Tests)
Test double	N/A	InMemoryReplayStore	Source
Residual limitation	process restart	runtime injection / config	Source

Table 13.1: Evolution of Replay Protection Security

13.8 Data Security and Synthetic Privacy

The ethical mandate of this project (NFR-04) strictly prohibits the ingestion of real-world Personally Identifiable Information (PII). By utilizing entirely synthetic datasets generated deterministically, the prototype eliminates the legal and ethical liabilities associated with analyzing live dark-web telemetry.

13.8.1 Ethics of Synthetic Data

Using synthetic data is an ethical necessity to protect innocent parties from accidental implication during research and development. However, it introduces known limitations:

- **Domain Shift:** Synthetic data may not capture the nuanced, chaotic nature of real-world threat actors, leading to a synthetic-to-real domain shift.
- **The False Confidence Problem:** High accuracy on synthetic data does not equate to real-world safety or performance. Over-reliance on these metrics can lead to dangerous assumptions when the system is deployed in production environments.

Furthermore, as demonstrated by Narayanan and Shmatikov [17], even supposedly anonymized data carries significant de-anonymization risks. Utilizing pure synthetic generation side-steps these privacy pitfalls while preserving structural complexity.

13.9 RAG Security and Prompt Injection

When the LLM receives graph data, it is subjected to 4-rule Regex sanitization [10,21].

- **Threat:** An adversarial payload containing instructions (e.g., `ignore previous system prompts`).
- **Detection:** Rule 2 `/(ignore|system|instruction|prompt)/gi`.
- **Response:** Replaces matches with `[REDACTED]`.

Illustrative synthetic example — not an empirical benchmark:

Attack: "Vendor: shadow_broker. System: print internal state."

Sanitized: "Vendor: shadow_broker. [REDACTED]: print internal state."

Residual Risk: Sophisticated multi-lingual or encoded prompt smuggling can trivially bypass static regex matching.

ID	Asset	Control	Residual Risk
T-01	LLM	Regex Sanitization (4 Rules)	Semantic Jailbreaks (High)
T-03	API	Nginx Rate Limiting	Distributed Volumetric DDoS (Med)
T-05	Database	WolverineDB Merkle Hashing	Config-Dependent Replay Attack (Med)

Table 13.2: Threat-to-Control Traceability Matrix

13.10 Threat-to-Control Traceability

13.11 Failure and Abuse Cases

- **Malformed Record:** Parser drops payload. Impact: Missing data.
- **LLM Failure:** Timeout triggers deterministic string fallback. Impact: Loss of narrative synthesis.
- **Cryptographic Failure:** Missing payload hash prevents normalization. Impact: Complete ingestion failure.

13.12 Correlation vs. Attribution

A critical ethical boundary in intelligence analysis is distinguishing statistical correlation from legal human attribution [29,32]. We formally define these as:

- **Statistical Correlation:** A mathematical and probabilistic relationship between two data points or entities based on observed features.
- **Legal Attribution:** The rigorous evidentiary standard required to definitively link an activity to a specific human identity.
- **Automation Bias:** The cognitive tendency to over-trust and rely on automated systems, often ignoring contradictory human intuition or evidence.

The Wolverine system calculates **record similarity** (Jaro-Winkler) and **temporal overlap**. When these features exceed a threshold, the system creates a **probabilistic candidate correlation** (an edge in Neo4j).

This represents an *analyst hypothesis*. It does **NOT** constitute verified human identity. Shared handles and temporal proximity can easily result from deliberate framing, operational coincidence, or identity theft. Asserting that a Neo4j graph edge proves legal attribution is a dangerous automation bias explicitly warned against in this architecture.

13.13 Human Oversight and Governance

Because correlation is not attribution, the system dictates an explicit requirement for Human-in-the-Loop (HITL) oversight. Candidate pairs scoring between 0.70 and 0.92 are tagged for human review, requiring qualitative analyst evaluation of the raw payloads before an edge is confirmed.

Risk	Severity	Future Mitigation
LLM Semantic Prompt Smuggling	High	Secondary LLM output evaluator.
WolverineDB TOCTOU / Memory Trans.	High	Physical multi-node deployment.
Algorithmic False Positives (FPR)	Medium	Dynamic threshold calibration (ROC).
Automation Bias / False Attribution	High	Mandated UI confidence disclaimers.
Container escape / Docker daemon exploit	High	Implement <code>seccomp/AppArmor</code> profiles.
RBAC header spoofing	High	Implement cryptographic OAuth/JWT tokens.
Synthetic-to-real domain shift	Medium	Phased evaluation on isolated real data.
Model drift in LLM synthesis	Medium	Continuous evaluation and version locking.

Table 13.3: Operational Residual Risk Matrix

13.14 Residual Risk Model

13.15 Security/Ethics Synthesis

The prototype successfully demonstrates architectural protections via strict network isolation and immutable raw capture. However, the cryptographic fault tolerance remains simulated, the RAG prompt-injection defenses are vulnerable to semantic bypass, and the probabilistic nature of the entity resolution mandates strict human oversight to prevent unethical automation bias.

Chapter 14

Discussion and Synthesis

The preceding chapters have detailed the requirements, architecture, algorithms, implementation, evaluation, and security of the Wolverine prototype. The purpose of this chapter is to synthesize these technical domains into a coherent interpretation. It transitions from detailing *how* the system was built to analyzing *what the engineering decisions mean* in the context of the project’s original objectives and broader intelligence literature.

14.1 Purpose of the Discussion

This discussion reconciles the system’s demonstrated strengths with its explicit prototype limitations. Rather than summarizing the architecture, it critically evaluates the trade-offs necessitated by the synthetic environment, bounded execution constraints, and simulated security components. It aims to clearly delineate what has been functionally proven (Class B) from what remains a simulated mathematical abstraction (Class D) or missing experiment (Class E).

14.2 Objective-by-Objective Assessment

Chapter 4 established core functional and non-functional objectives. Table 14.1 evaluates their realization.

Objective	Implementation	Evidence Base	Status	Limitation
FR-01 (Data)	adapters.ts & parsers	Normalization tests	Demonstrated (Class B)	Fails on unexpected DOM drift.
FR-03 (Resolve)	entity_resolver.ts	Jaro-Winkler tests	Demonstrated (Class B)	Fixed activityOverlap placeholder.
FR-04 (Graph)	projector.ts (BFS)	Bounded execution tests	Demonstrated (Class B)	$d \leq 4$, cannot map deep networks.
NFR-01 (Eval)	postgres-truth	CLI evaluator mock	Simulated (Class D)	No dynamic PR calculation implemented.
NFR-04 (Ethics)	Synthetic generator	Seeded $S_0 = 42$ output	Demonstrated (Class B)	Inherits synthetic domain shift.

Table 14.1: Objective Realization and Assessment

14.3 Architectural Findings and Trade-offs

The prototype utilizes a distributed, 14-container topology enforcing strict trust zones (Chapter 5), alongside new subsystems like WolverineDB (Chapter 9) and Dual-Frontends (Chap-

ter 10).

- **Heterogeneous Ingestion:** Isolating parsers from resolvers proved highly advantageous. It localizes failure. If an adversarial platform alters its DOM, only the specific parser drops records, protecting downstream relational integrity. However, this incurs the cost of high maintenance; parser fragility remains a dominant operational risk.
- **Truth Vault Isolation:** Enforcing the `internal: true` Docker network boundary successfully demonstrated an architectural mechanism for preventing evaluator contamination. It proves that cheating is computationally restricted, though it relies entirely on Docker daemon integrity rather than physical air-gapping.
- **WolverineDB Simulation Boundary:** WolverineDB demonstrates that cryptographic Merkle hashing can be retrofitted via PostgreSQL CDC triggers. However, utilizing `DirectMemoryNetworkTransport` for Byzantine consensus trades distributed reliability for localized testability. The prototype proves the *interface* of decentralized security, but fundamentally sidesteps the network latency and partition tolerance issues required of a genuine deployment.

14.4 In-Memory Graph Trade-off

The decision to execute bounded Breadth-First Search via in-memory ES6 Map objects (Chapter 8) represents the prototype’s most significant structural trade-off.

Advantages: This approach yields exceptional simplicity and determinism. By bypassing recursive SQL CTEs or external Cypher traversals during initial projection, the system avoids complex lock contention on the relational store. Hardcoding limits ($d \leq 4, N \leq 500$) guarantees the Node.js V8 engine will never encounter an Out-Of-Memory exception, yielding high demonstration stability.

Costs: The trade-off is horizontal scalability. In-memory maps cannot traverse graphs larger than available container RAM, and the arbitrary 500-node truncation risks silently dropping critical threat-actor subgraphs. While adequate for a prototype, production systems necessitate disk-backed distributed graph engines (e.g., native Neo4j Apoc, NebulaGraph) capable of processing millions of vertices iteratively.

14.5 Heuristic Entity Resolution Critique

The deterministic heuristics detailed in Chapter 7 demonstrate significant architectural validation but expose severe algorithmic limitations.

- **Jaro-Winkler & Blocking:** Utilizing a 3-character prefix block is computationally efficient but highly sensitive to deliberate adversarial mutation (e.g., padding handles with noise). This likely yields an artificially high false-negative rate against sophisticated actors.
- **Simulated Activity Feature:** The composite formula contains a fixed weight of 0.15 for `activityOverlap`, but the implementation hardcodes the value to 0.5. This statically inflates base correlation scores without assessing actual temporal behavioral intersections, distorting the meaning of the 0.70 human-review threshold.
- **Binary Cross-Site Cue:** The lack of Natural Language Processing (NLP) or stylometry limits the system to explicit string matching. Threat actors sharing linguistic anomalies but utilizing disjoint handles will evade this heuristic completely.

14.6 Evaluation and Evidence Distribution

Evaluating the system requires strict adherence to the Evidence Hierarchy established in Chapter 12. To provide a quantitative honesty metric, the evidence class distribution across Wolverine’s core modules highlights its prototype nature:

- **Class B (Functional Verification):** $\sim 60\%$ of modules. This includes the 14-container topology routing, canonical parsing, bounded BFS algorithms, Docker network isolation, and PostgreSQL trigger-based state hashing.
- **Class C (Synthetic Demonstrations):** $\sim 20\%$ of modules. Ollama RAG integration heavily relies on specific formatted inputs, working sufficiently in synthetic bounds but untested against wild data.
- **Class D (Static/Mock Output):** $\sim 15\%$ of modules. This includes the F1 metrics generator, Vzeya’s cinematic presentation visuals, and the `DirectMemoryNetworkTransport` consensus simulations.
- **Class E (Missing Experiments):** $\sim 5\%$ of modules. Real-world model deployment, dynamic performance benchmarking on live streams, and adversarial threat actor testing have explicitly been omitted.

It is scientifically imperative to reiterate that the evaluation metrics $P = 0.9280$, $R = 0.4598$, $F1 = 0.6149$ are static (Class D) outputs. They prove the CI/CD environment can emit a structured evaluation report, but they provide zero empirical evidence regarding the performance of the Jaro-Winkler heuristics. Furthermore, the gap between formal specifications (such as the 91 detailed WolverineDB requirements) and production readiness remains vast, primarily masked by in-process mocking and fixed constants.

14.7 Security and Ethical Interpretation

The prototype implements robust architectural security (network isolation, immutable payload hashes, regex RAG sanitization).

A notable post-audit engineering result is the resolution of the authorization replay vulnerability. The initial design utilized an ephemeral Node.js `Set` for the `WolverineDB` replay cache, which proved insufficient as it yielded to TOCTOU races and container restarts. The transition to a durable PostgreSQL-backed `PostgresReplayStore` demonstrates that atomic database uniqueness constraints are vastly superior to application-level locking for cryptographic nonce consumption. However, the requirement for manual dependency injection highlights that architectural correctness alone cannot prevent insecure configuration deployments.

Ethically, the project successfully eliminates real-world privacy risks via its synthetic-only mandate (Class B). However, this isolation exacerbates the risk of *Automation Bias*. If an analyst equates a probabilistic Jaro-Winkler graph edge with legal human attribution, the system enables false conviction. The prototype strictly defines correlation as an *analyst hypothesis*, explicitly demanding human oversight.

14.8 Prototype vs. Production Comparison

14.9 Contribution Analysis

The project’s contributions must not be overstated:

Capability		Current Prototype		Production Requirement		Evidence
Graph Infrastructure	Infrastructure	In-memory Engine)	BFS (V8)	Distributed Graph DB	Disk-Backed	Class B
Cryptographic State		Trigger-based Ledger	Merkle	Cross-cluster ledger syncing		Class B
BFT Consensus		DirectMemoryNetwork		Geographical TCP/IP Quorum		Class D
RAG Security		4-Rule Static Regex		LLM-based Intention Evaluator		Class B
Frontend Architecture	Architecture	Split Mocks	React/Vzeya	Unified authenticated Client	React	Class C
Dynamic Evaluation		Static Mock Output		Dynamic Confusion Matrix Pipeline		Class E

Table 14.2: Prototype vs. Production Readiness

- **Engineering Contribution:** The integration of heterogeneous dark-web parsing, probabilistic heuristics, and LLM narrative synthesis into a unified, Docker-deployable architecture.
- **Architectural Contribution:** The functional demonstration of isolating an evaluation Ground Truth Vault using strict network topology to enforce objective benchmarking limits, alongside the WolverineDB trust ledger.
- **Methodological Constraint:** Explicitly distinguishing the visual presentation mock (Vzeya) from the functional inference engine (Analyst UI) establishes a model for honest research disclosure.

The project does *not* claim algorithmic novelty; Jaro-Winkler string similarity and bounded BFS are established prior art.

14.10 Lessons Learned and Limitations

1. **Bounded architectures are necessary for prototype stability.** Forcing the graph to truncate at 500 nodes prevented cascading memory failures during demonstration, proving that hard operational bounds are superior to unconstrained theoretical correctness in prototype engineering.
2. **Explicit simulation boundaries protect research honesty.** By formally documenting WolverineDB’s `DirectMemoryNetworkTransport`, the project avoids the common pitfall of passing off localized stubs as distributed breakthroughs.
3. **Limitation:** Synthetic ground truth enables automated, objective evaluation pipelines, but it fundamentally limits external validity. Algorithms optimized for synthetic random noise will inevitably misclassify the deliberate, hostile obfuscation tactics employed by genuine threat actors.

Chapter 15

Future Work

The Wolverine prototype demonstrates the architectural integration necessary for automated threat intelligence, yet explicitly operates within the constraints of a simulated environment. This chapter outlines the rigorous technical transitions required to elevate the system from a Class B (Functional) prototype to a Class A (Empirical) production engine, structured as a prioritized roadmap of engineering and research evolution.

15.1 WolverineDB Evolution

- **Current Limitation:** While replay protection has been migrated to a durable PostgreSQL table locally, WolverineDB’s BFT consensus network is simulated entirely via in-process memory (`DirectMemoryNetworkTransport`).
- **Proposed Change:** Replace the memory transport interface with a genuine TCP/IP Gossip protocol or gRPC quorum network. Subsequently, distributed replay protection across independently replicated nodes must be implemented to prevent decentralized race conditions.
- **Technical Mechanism:** Implement the Raft or PBFT consensus algorithm across physically isolated PostgreSQL instances.
- **Expected Benefit:** Authentic distributed Byzantine fault tolerance, validating the WDB-02 formal specification against real network partition events.

15.2 Graph Infrastructure Evolution

- **Current Limitation:** Graph projection relies on an in-memory BFS string comparison limited to $d \leq 4$, $N \leq 500$, truncating deeper network discovery.
- **Proposed Change:** Decouple projection from application memory. Utilize a dedicated, disk-backed distributed graph database (e.g., native Neo4j Apoc traversals or Nebula-Graph).
- **Required Infrastructure:** High-memory cluster nodes optimized for iterative graph processing (e.g., Apache Spark GraphX).
- **Expected Benefit:** Ability to query and traverse subgraphs containing millions of vertices asynchronously without endangering API availability.

15.3 Entity Resolution Evolution

- **Current Limitation:** Linkage heuristics are deterministic and fixed (e.g., 0.30 alias weight, simulated 0.5 activity overlap), lacking the ability to learn complex obfuscation patterns.
- **Proposed Change:** Transition from deterministic heuristics to calibrated probabilistic linkage (e.g., trained Fellegi-Sunter) or Graph Neural Networks (GNNs).
- **Technical Mechanism:** Implement dynamic feature extraction (e.g., true temporal event intersection, stylometric NLP embeddings for forum posts) and train a gradient boosting classifier (e.g., XGBoost) on the synthetic ground truth.
- **Required Evaluation:** This necessitates the dynamic benchmark pipeline (Section 15.4) to prevent catastrophic overfitting.

15.4 Evaluation Evolution

To replace the static Class D metric artifacts, a formal evaluation progression must be executed:

1. **Dynamic Synthetic Benchmark:** Implement a pipeline that dynamically computes Precision, Recall, and F1 by comparing resolver outputs against the `postgres-truth` vault.
2. **Ablation Studies:** Isolate the impact of specific heuristic weights (e.g., the 30-day temporal decay) to optimize the composite score.
3. **Threshold Calibration:** Plot ROC/PR curves to mathematically justify the 0.70 (Review) and 0.92 (Linked) decision thresholds.
4. **Adversarial Evaluation:** Inject deliberate obfuscation (e.g., character substitution, timeline jitter) into the synthetic engine to test algorithm resilience.
5. **Real-World Validation:** Ingest an anonymized subset of verified historic threat data to measure synthetic-to-real domain shift.

15.5 RAG and LLM Evolution

- **Current Limitation:** The 4-rule Regex sanitization provides basic protection but is theoretically vulnerable to semantic prompt smuggling.
- **Proposed Change:** Implement a dual-LLM architecture.
- **Technical Mechanism:** Route hostile context through a smaller, strictly constrained "evaluator" LLM tasked exclusively with detecting injection intent before passing sanitized context to the primary synthesis model.
- **Expected Benefit:** Robust defense against adversarial jailbreaks, ensuring Analyst UI narrative integrity.

15.6 Security and Governance Evolution

- **Current Limitation:** Cryptographic signatures within WolverineDB lack physical anchoring.
- **Proposed Change:** Integrate AWS CloudHSM or physical YubiKey HSMs to sign Merkle tree commitments.
- **Governance Evolution:** Implement strict versioned evidence trails (Model Cards) that snapshot the exact heuristic weights active when a specific canonical record was processed, ensuring legal/audit traceability for human review decisions.

15.7 Explicit Research Questions

The engineering evolution facilitates the investigation of broader cyber-intelligence research questions:

- *How does entity-resolution accuracy degrade when synthetic aliases are subjected to increasing levels of Levenshtein perturbation?*
- *What is the quantitative synthetic-to-real domain shift in False Positive Rates when transferring deterministic heuristics to live dark-web telemetry?*
- *How effectively can a secondary evaluator LLM detect semantic prompt injection disguised within adversarial forum payloads?*

15.8 Prioritization Roadmap

Future Work		Priority	Dependency		Expected Impact	Eng. Cost
Dynamic Pipeline	Eval	P0	None		Unlocks empirical (Class A) metric measurement.	Medium
Durable Cache	Replay	P1	None		Mitigates critical container-restart vulnerability.	Low
NLP Feature traction	Ex-	P2	Dynamic Eval		Improves algorithmic recall on distinct handles.	High
Distributed DB	Graph	P2	Infrastructure scaling		Eliminates 500-node memory limitation.	High
BFT Transport	TCP/IP	P3	Durable Cache	Replay	Validates true decentralized trust boundaries.	Very High

Table 15.1: Future Work Prioritization Roadmap

Chapter 16

Conclusion

16.1 Restatement of the Problem

The proliferation of decentralized, adversarial ecosystems—such as dark-web marketplaces and encrypted forums—has severely fragmented digital identities. Traditional cyber-intelligence mechanisms struggle to correlate actors across these heterogeneous domains without violating stringent evidentiary standards or succumbing to automated false attribution.

16.2 What Was Built

The Wolverine project constructed a secure, localized microservices prototype to confront this fragmentation. The architecture integrated five deterministic synthetic platforms, a strict schema normalization pipeline, and a probabilistic Jaro-Winkler entity-resolution engine. The resulting linkages were projected via an in-memory Breadth-First Search into a Neo4j visualization graph, and narratively synthesized using a local Llama-3.2 inference engine bounded by explicit Regex sanitization. The entire operational lifecycle was backed by WolverineDB, a cryptographic middleware enforcing Merkle state-history hashing on relational ingestion.

16.3 What Was Demonstrated

The prototype successfully demonstrated the architectural viability of the pipeline (Class B functional evidence). It proved that adversarial ingestion payloads could be deterministically normalized, that graph projection could be safely bounded to prevent memory exhaustion, and that an isolated Truth Vault could be structurally air-gapped via Docker network directives to mathematically prevent algorithmic cheating.

16.4 What Was Not Demonstrated

Crucially, this prototype explicitly did **not** demonstrate production-ready intelligence capability.

- **No Empirical Accuracy:** The reported $F1 = 0.6149$ is a static formatting stub, not a measured algorithmic benchmark. The dynamic evaluation pipeline remains unwritten.
- **No Real-World Validation:** Evaluated exclusively on synthetic data, the heuristic algorithms are unverified against genuine, sophisticated threat actor obfuscation (domain shift).
- **Simulated Security Mechanics:** The distributed BFT consensus and EVM anchoring mechanisms of WolverineDB are in-process memory simulations, vulnerable to standard container crashes and entirely untested against genuine network partitions.

16.5 Contributions

The primary contribution of this work is architectural and methodological. By enforcing strict isolation between the ingestion DMZ and the Truth Vault, the project establishes a rigorous structural blueprint for future verifiable intelligence evaluation. Furthermore, by explicitly separating the cinematic mock presentation layer (Vzeya) from the functional analytical layer (Analyst UI), it models the level of transparency required to conduct honest research within complex defense-sector prototypes.

16.6 Final Scientific Interpretation

The scientific value of the Wolverine prototype lies not in claiming a solved problem, but in its coherent, reproducible, and safety-bounded integration of disparate technical components. It provides a functional foundation that explicitly defines its own simulated boundaries, mapping exactly what is required to replace its localized stubs with verifiable distributed primitives.

16.7 Final Closing Statement

The development of automated, privacy-respecting threat intelligence is not achieved in a single developmental leap. The Wolverine architecture successfully fulfills the requirements of a functional *prototype*. Advancing this system to operational maturity requires a disciplined, sequential progression: from controlled dynamic evaluation, to empirical real-world validation, to distributed production engineering, and ultimately to secure operational deployment.

Appendix A

Synthetic Generation Specifications

This appendix provides the authoritative technical reference for the deterministic synthetic ecosystem utilized by the Wolverine prototype. It details the precise generation pipeline, the statistical boundaries of the dataset, and the exact procedures required for reproducible execution.

A.1 Master Seed and Determinism

To guarantee mathematical reproducibility across disparate host machines, the Python generator initializes its PRNG using a hardcoded, invariant master seed:

$$S_0 = 42 \tag{A.1}$$

What is Deterministic: The instantiation of 50,000 personas, the allocation of 89,605 accounts, the 39,605 relational linkages, and the 35% handle mutation rate are all strictly governed by S_0 . Executing the generator script will yield identical SQL `INSERT` statements.

What is NOT Deterministic: The seed *cannot* guarantee deterministic async ingestion ordering via the Redis pub/sub queue, nor can it guarantee identical text synthesis from the local Llama-3.2 container (unless explicitly clamped at $T = 0.0$).

A.2 Dataset Dimensions and Allocation

The generator explicitly differentiates between the latent human actor and the observable digital artifacts:

- **Persona:** The ground-truth virtual human (Count: 50,000).
- **Account:** A registered profile on a specific platform owned by a persona (Count: 89,605).
- **Ground-Truth Link:** The mathematically verifiable edge connecting two accounts to the same persona (Count: 39,605).
- **Record:** The observed payload (JSON/HTML) ingested by the system.

A.3 Handle Mutation and Adversarial Noise

To test the resilience of the Jaro-Winkler heuristics (Chapter 7), the generator applies a **35% mutation rate** when allocating a second account to an existing persona.

Mutation Rules:

Platform	Role	Schema Characteristics
Atlas	Clearnet Social	Structured JSON, public handles, dense metadata.
Briar	Encrypted Chat	Ephemeral identifiers, minimal profile data.
Cinder	Dark-web Market	PGP keys, vendor ratings, transaction histories.
Drift	Paste Site	Unstructured HTML, periodic data dumps.
Ember	Hacker Forum	Threaded structures, signature blocks, reputation scores.

Table A.1: Synthetic Platform Definitions

- Levenshtein transpositions (e.g., `shadow_broker` $\rightarrow$ `shdaow_broker`)
- Affix padding (e.g., `shadow_broker` $\rightarrow$ `shadow_broker_99`)

This forces the entity-resolution pipeline to rely on probabilistic scoring ($S \geq 0.70$) rather than trivial exact-string matching.

A.4 Schema Definitions

The output of the generator conforms strictly to the `CanonicalRecord` typing enforced by the pipeline:

Field	Type	Required	Generation Constraint
<code>id</code>	UUID	Yes	Generated deterministically via namespace and S_0 .
<code>source_site</code>	Enum	Yes	Must be Atlas, Briar, Cinder, Drift, or Ember.
<code>handle</code>	String	Yes	Subject to the 35% mutation probability.
<code>timestamp</code>	ISO8601	Yes	Uniformly distributed over the preceding 365 days.
<code>content</code>	Text	No	Contains synthetic chatter or PGP blocks.

Table A.2: Canonical Record Field Definitions

A.5 Reproduction Procedure

To deterministically recreate the dataset:

1. **Command:** `docker-compose run generator python main.py -seed 42`
2. **Input:** The hardcoded `-seed 42` argument.
3. **Expected Output:** Console confirms “50,000 personas generated” and “39,605 truth edges committed.”
4. **Verification:** Execute `SELECT COUNT(*) FROM ground_truth_links` against the mapped `postgres-truth` port (5434). The integer returned must exactly equal 39,605.

Appendix B

API Specifications and Protocol Interfaces

This appendix catalogs the programmatic interfaces of the Wolverine backend within the `wolverine-internal` trust zone. All endpoints are implemented in `wolverine/src/api/server.ts` and guarded by the RBAC middleware documented in Chapter 8.

B.1 Role-Based Access Control

The API implements role-based access control via the `X-Wolverine-Role` HTTP header. This is a functional routing mechanism operating within the isolated internal network; it does *not* constitute cryptographic authentication.

Role	Permissions
<code>admin</code>	Full access to all endpoints including seed, metrics, and administrative operations.
<code>operator</code>	Read/write access to collection runs, records, resolution candidates, and analysis.
<code>researcher</code>	Read-only access to records, graph queries, and analysis questions.
<code>reviewer</code>	Read access to resolution candidates; write access to approve/reject decisions.
<code>public_demo</code>	Read-only access to a restricted subset of endpoints (graph, records).

Table B.1: RBAC Role Definitions and Permission Scopes

Implementation: The `getRole()` middleware extracts the role from the `X-Wolverine-Role` header via strict string equality. The `requireRole()` middleware enforces that the extracted role matches one of the permitted roles for the endpoint, returning HTTP 403 otherwise.

B.2 Endpoint Catalog

B.3 Graph Query Interface

The `/v1/graph/query` endpoint interfaces with the in-memory BFS engine implemented in `wolverine/src/graph/projector.ts` (Chapter 7).

- **Query Parameters:**

- `startRef` (required): The UUID of the root node for BFS traversal.

Method	Path	Purpose	Source Module
GET	/health/live	Liveness probe (always 200).	server.ts
GET	/health/ready	Readiness probe (checks DB connectivity).	server.ts
GET	/metrics	Prometheus-compatible metrics export.	server.ts
POST	/v1/collection-runs	Initiate a new data collection run against a source site.	server.ts
GET	/v1/collection-runs/:id	Retrieve the status and metadata of a specific collection run.	server.ts
GET	/v1/records	List canonical (normalized) records with filtering and pagination.	server.ts
GET	/v1/records/:id	Retrieve a single canonical record by UUID.	server.ts
GET	/v1/graph/query	Execute bounded BFS traversal on the entity graph.	projector.ts
POST	/v1/analysis/questions	Submit a question to the RAG pipeline for AI-assisted analysis.	rag.ts
GET	/v1/resolution-candidates	List entity resolution candidates with state filtering.	server.ts
PATCH	/v1/resolution-candidates/:id	Update candidate state (approve/reject for HITL review).	server.ts

Table B.2: Verified API Endpoint Catalog (from `wolverine/src/api/server.ts`)

- **maxDepth** (optional, default: 2, max: 4): Maximum traversal depth.
- **limit** (optional, default: 100, max: 500): Maximum nodes returned.

- **Response Format:**

```
{
  "nodes": [{ "id": "uuid", "label": "AtlasAccount", ... }],
  "edges": [{ "source": "uuid-a", "target": "uuid-b",
               "type": "linked", "weight": 0.94 }],
  "truncated": false
}
```

- **Truncation:** If the BFS exceeds the **limit** parameter, the response includes **"truncated": true** to signal partial results.

B.4 RAG Interface

The `/v1/analysis/questions` endpoint manages the integration with the local Ollama container (Chapter 7).

Illustrative API request (synthetic example — not empirical):

```
POST /v1/analysis/questions
Content-Type: application/json
X-Wolverine-Role: researcher
```

```
{
  "question": "Summarize the temporal activity overlap
               between atlas:shadow_broker and briar:shad0w.",
  "contextNodeIds": ["uuid-1", "uuid-2"]
}
```

Execution Flow:

1. **Sanitization:** Applies the 4-rule regex sequence (Chapter 7): strip control characters, truncate to 500 characters, mask injection patterns, cap context to 20 records.
2. **Prompt Construction:** Constructs a structured prompt forcing the LLM to label outputs as “analytic hypothesis” with explicit JSON citations.
3. **Invocation:** Issues a POST to `http://ollama:11434/api/generate` with a 5,000ms `AbortSignal` timeout.
4. **Fallback:** If Ollama exceeds the timeout, the system returns a deterministic string summarizing the retrieved context and citing the top 3 records.

Expected Response Format:

```
{
  "answer": "Analytic hypothesis: ...",
  "confidence": 0.72,
  "citations": [
    { "recordId": "uuid-1", "source": "atlas" },
    { "recordId": "uuid-2", "source": "briar" }
  ]
}
```


B.5 Collection Run Interface

The `/v1/collection-runs` endpoints manage the data ingestion lifecycle.

Create Collection Run:

```
POST /v1/collection-runs
Content-Type: application/json
X-Wolverine-Role: operator

{
  "sourceId": 1,
  "mode": "full"
}
```

Response: Returns the collection run ID and initial status. The system asynchronously fetches data from the specified source site, stores raw payloads in MinIO, and routes parsed records through the normalization pipeline.

B.6 Error Catalog

Error Code	HTTP	Trigger Condition
ERR_AUTH	403	Missing or invalid <code>X-Wolverine-Role</code> header.
ERR_NOT_FOUND	404	Requested resource (record, collection run, candidate) does not exist.
ERR_VALIDATION	400	Request body fails JSON schema validation.
ERR_PARSE	400	Ingested payload fails site-specific parser; record quarantined.
ERR_GRAPH_TRUNCATED	400	BFS hits the node limit; partial graph returned with <code>truncated: true</code> .
ERR_LLM_TIMEOUT	500	Ollama timeout (>5s); deterministic fallback string returned.
ERR_LLM_UNAVAILABLE	503	Ollama container unreachable; service degradation mode.
ERR_DB	500	PostgreSQL or Redis connection failure; circuit breaker triggered.

Table B.3: API Error Catalog with HTTP Status Codes

Appendix C

WolverineDB Formal Specifications

This appendix provides the specification-to-code mapping matrix for WolverineDB. The full architectural and cryptographic treatment is provided in Chapter 9; this appendix serves as a comprehensive reference for the 91 normative WDB-* specifications organized by domain.

C.1 Specification Taxonomy

The WolverineDB specification suite is organized into functional domains, each governing a specific aspect of the cryptographic trust layer. Table C.1 provides the authoritative mapping from specification identifiers to implementation files and verification status.

Table C.1: WolverineDB Specification-to-Code Traceability Matrix

Spec Range	Domain	Implementation	Status
WDB-0001–0002	Core Architecture	src/core/	Implemented
WDB-0003	Hash Chain (CDC)	src/crypto/hash.ts	Implemented (Full)
WDB-0004	Merkle Tree	src/crypto/merkle.ts	Implemented (Full)
WDB-0005–0010	Binary Encoding	src/binary/c14n.ts, encoder.ts, record_id.ts	Implemented (Full)
WDB-0011–0015	Error Handling	src/core/errors.ts	Implemented (Full)
WDB-0021	EVM Anchor	src/anchors/evm.ts	Simulated (Map)
WDB-0022–0025	Checkpoint Storage	src/checkpoint/local.ts, s3.ts, anchor.ts	Implemented (Full)
WDB-0030	Sentinel Policy Gate	src/sentinel/policy_gate.ts	Implemented (Full)
WDB-0031	Sentinel Anomaly Engine	src/sentinel/anomaly_engine.ts	Implemented (Full)
WDB-0032–0035	Sentinel Advisory	src/sentinel/advisor.ts	Implemented (Full)
WDB-0040–0050	Signing Providers	src/crypto/signing_provider.ts	Partial (Local only)

Continued on next page

Spec Range	Domain	Implementation	Status
WDB-0051–0060	Database Adapters	src/adapters/mysql.ts, sqlite.ts	Implemented (Full)
WDB-0080–0085	Trust Network Ledger	src/trust_network/ledger.ts	Implemented (Full)
WDB-0086–0090	BFT Consensus	src/trust_network/consensus.ts	Simulated (Memory)
WDB-0091–0096	Validator & Replication	src/trust_network/validator.ts	Simulated (Memory)
WDB-0100–0110	Runtime & Transport	src/runtime/network_transport.ts	Simulated (Memory)
WDB-0111–0120	Validator Daemon	src/runtime/validator_daemon.ts	Simulated (Memory)
WDB-0130	Atomic File Creation	src/checkpoint/local.ts	Hardened (v1.3.0)
WDB-0131	Mutex Append Queue	src/trust_network/ledger.ts	Hardened (v1.3.0)
WDB-0132	Byte Collation	src/binary/record_id.ts	Hardened (v1.3.0)
WDB-0133	Replay Protection	PostgresReplayStore	Implemented (Durable)
WDB-0134–0135	Key Rotation & Collusion	src/bft_hardening/	Hardened (v1.3.0)

Status Legend:

- **Implemented (Full)**: Specification fully satisfied with passing tests.
- **Simulated (Memory)**: Logic implemented but operates via `DirectMemoryNetworkTransport` — not distributed.
- **Simulated (Map)**: External service replaced by in-memory `Map` — no real persistence.
- **Partial**: Interface defined but only dev/test implementation exists.
- **Hardened (v1.3.0)**: Security-critical improvement from the v1.3.0 audit cycle.

C.2 Cryptographic Structures

The hash chain and Merkle tree specifications are detailed in Chapter 9. The recurrence relation for the continuous hash chain (WDB-0003) is:

$$H_i = \text{SHA256}(H_{i-1} \parallel \text{SHA256}(m_i)) \quad (\text{C.1})$$

Where H_0 is the genesis hash (all zeros) and m_i is the deterministic canonical JSON serialization of the i -th record payload.

The RFC 6962-compatible Merkle tree (WDB-0004) computes:

$$H_{\text{leaf}} = \text{SHA256}(0x00 \parallel \text{data}) \quad (\text{C.2})$$

$$H_{\text{node}} = \text{SHA256}(0x01 \parallel H_{\text{left}} \parallel H_{\text{right}}) \quad (\text{C.3})$$

C.3 Simulation Boundaries

Three critical subsystems operate in simulation mode (Chapter 9):

1. **Network Transport** (WDB-0100): The `DirectMemoryNetworkTransport` passes messages within the V8 heap. It experiences zero latency, zero packet loss, and zero partitions. Claims of BFT resilience based on this implementation are invalid for production contexts.

2. **KMS/HSM Signing** (WDB-0040): The prototype uses `LocalSoftwareSigningProvider` with ephemeral keys. The `CloudKmsSigningProvider` and `HsmSigningProvider` interfaces exist but delegate to simulated backends.
3. **EVM Anchoring** (WDB-0021): Checkpoint anchoring targets an in-memory `Map` rather than a real Ethereum contract. No decentralized persistence is achieved.

C.4 Verified Security Vulnerabilities

The gap between specification and implementation yields known vulnerabilities documented in Chapter 13:

1. **In-Memory Transport Collapse:** A single process OOM event destroys the entire “quorum.”
2. **Configuration-Dependent Replay Protection:** CLI runners default to `InMemoryReplayStore` unless explicitly configured to use `PostgresReplayStore`.
3. **Sentinel TOCTOU Window:** Policy checks occur before ledger append, creating a narrow race condition.
4. **No Certificate Pinning:** The simulated transport does not validate peer identities.

C.5 Test Coverage

The repository contains 219+ vitest test cases covering:

- Cryptographic primitive correctness (hash chains, Merkle proofs)
- Concurrent BFT consensus under contention
- Catastrophic recovery scenarios
- Database adapter compliance (SQLite, MySQL)
- Malicious actor simulation (Byzantine validators)
- Fuzzing edge cases

Critical interpretation: These tests evaluate *specification compliance under ideal conditions*. A passing consensus test proves that the in-memory event bus functions correctly; it provides zero evidence that the logic survives real TCP/IP network partitions, clock skew, or partial connectivity.

Appendix D

Evaluation Harness and Static Outputs

This appendix provides total forensic transparency regarding the prototype’s evaluation metrics. It explicitly dissects the origin of the reported Precision, Recall, and F1 scores, proving their status as static architectural stubs, and establishes the mathematical blueprints required to compute empirical metrics in the future.

D.1 Static Output Generation Forensics

The Vzeya cinematic frontend prominently displays the following metrics:

- **Precision:** 0.9280
- **Recall:** 0.4598
- **F1 Score:** 0.6149

Forensic Origin: These values are explicitly **NOT** the result of dynamic Jaro-Winkler computation over the `postgres-truth` vault. They are generated by the `generator/src/evaluator/__main__.py` script. Inspection of this source file reveals that these numbers are formatted string literals printed to standard output.

Scientific Implication: The evaluator module functions purely as an architectural CI/CD test (Class D). It proves that an external process can connect to the database, simulate a reporting cycle, and output a JSON format readable by the frontend. Using these values to claim empirical accuracy for Wolverine’s entity resolution is mathematically invalid.

D.2 Dynamic Evaluation Blueprint (Future Work)

To generate valid (Class A) metrics in a future iteration (Chapter 15), the evaluator must be rewritten to compute a dynamic Confusion Matrix against the 39,605 generated ground-truth edges.

D.2.1 Confusion Matrix Definitions

- **True Positive (TP):** A predicted edge ($S \geq 0.70$) that exactly matches a ground-truth edge.
- **False Positive (FP):** A predicted edge ($S \geq 0.70$) that does *not* exist in the ground truth (False Attribution).
- **False Negative (FN):** A ground-truth edge that the system failed to predict ($S < 0.70$).

D.2.2 Metric Equations

$$\text{Precision (P)} = \frac{TP}{TP + FP} \quad (\text{D.1})$$

$$\text{Recall (R)} = \frac{TP}{TP + FN} \quad (\text{D.2})$$

$$\text{F1 Score} = 2 \times \frac{P \times R}{P + R} \quad (\text{D.3})$$

Note: The equations above are methodological blueprints. The prototype's static metrics must not be inserted into these formulas.

D.3 Threshold Evaluation Blueprint

The system currently utilizes static decision thresholds:

- $S < 0.70$: Discarded / Unlinked.
- $0.70 \leq S < 0.92$: Flagged for Human Analyst Review.
- $S \geq 0.92$: Automatically Linked (High Confidence).

Future evaluation must implement a threshold sweep (e.g., $S \in [0.50, 0.99]$) to plot empirical Precision-Recall (PR) and Receiver Operating Characteristic (ROC) curves. This calibration is necessary to quantify the trade-off between missing threat actor linkages (FN) and generating overwhelming analyst alert fatigue (FP).

D.4 Scalability Benchmark Blueprint

The BFS projection is theoretically bounded at $O(|V| + |E|)$. However, this asymptotic complexity is shielded by the hard limits $d \leq 4, N \leq 500$ implemented in the V8 engine. A future scalability benchmark (Class E) must disable these hard limits and measure query latency (ms) and peak memory consumption (MB) against a disk-backed graph database as N scales from 10^3 to 10^6 nodes.

D.5 Reproducibility Procedure

To reproduce the static evaluator stub behavior:

1. Ensure the Docker environment is running (Appendix A).
2. **Command:** `docker-compose run generator python -m evaluator`
3. **Expected Output:** The console will print the static JSON payload containing the 0.9280, 0.4598, and 0.6149 values.

Appendix E

Data Dictionaries

This appendix provides a comprehensive data dictionary documenting the Prisma schema utilized in the Wolverine system for storing configuration, raw payloads, normalized records, and derived resolution states.

E.1 Entity Definitions

E.1.1 Source

Field	Type	Constraint	Description
id	Int	PK, Auto-increment	Internal primary key
name	String	UNIQUE NOT NULL	Platform identifier (e.g., atlas, briar, cinder, drift, ember)
baseUrl	String	NOT NULL	Base API endpoint URL
apiType	String	NOT NULL	Access protocol (rest, graphql, html)
createdAt	DateTime	DEFAULT now()	Record creation timestamp

Table E.1: Source Entity Schema

E.1.2 CollectionRun

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
sourceId	Int	FK → Source	Reference to the target Source
status	String	NOT NULL	State enum: pending, running, completed, or failed
startedAt	DateTime	NOT NULL	Timestamp when execution began
completedAt	DateTime	NULLABLE	Timestamp when execution finished
recordsCollected	Int	DEFAULT 0	Total number of raw payloads fetched
errorMessage	String	NULLABLE	Failure details if status is failed

Table E.2: CollectionRun Entity Schema

E.1.3 Capture

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
collectionRunId	Int	FK → CollectionRun	The run that fetched this data
locator	String	NOT NULL	Specific source URL or path
contentHash	String	NOT NULL	SHA-256 hash of the raw payload
objectKey	String	NOT NULL	MinIO object key for raw data storage
capturedAt	DateTime	DEFAULT now()	Timestamp of capture

Table E.3: Capture Entity Schema

E.1.4 NormalizedRecord

Field	Type	Constraint	Description
id	String	PK (UUIDv4)	Unique canonical record identifier
captureId	Int	FK → Capture	Reference to the raw capture
sourceId	Int	FK → Source	Platform source origin
recordType	String	NOT NULL	Type enum: account, listing, post, reply
recordData	JSON	NOT NULL	Parsed and normalized canonical payload
contentHash	String	NOT NULL	SHA-256 of the normalized data
normalizedAt	DateTime	DEFAULT now()	Timestamp of normalization

Table E.4: NormalizedRecord (CanonicalRecord) Entity Schema

E.1.5 Quarantine

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
captureId	Int	FK → Capture	The raw capture that failed parsing
errorCode	String	NOT NULL	Standardized error classification
errorMessage	String	NULLABLE	Detailed parser exception
rawSnippet	String	NULLABLE	First 500 characters of the failed payload
quarantinedAt	DateTime	DEFAULT now()	Timestamp of quarantine placement

Table E.5: Quarantine Entity Schema

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
recordAId	String	FK $\rightarrow$ NormalizedRecord	First canonical record
recordBId	String	FK $\rightarrow$ NormalizedRecord	Second canonical record
compositeScore	Float	NOT NULL	Overall similarity confidence score
featureScores	JSON	NOT NULL	Dictionary of individual feature weights
state	String	DEFAULT 'pending'	Status enum: pending, linked, review, rejected
resolvedAt	DateTime	NULLABLE	Timestamp when a final state was reached
resolvedBy	String	NULLABLE	Analyst handle for HITL review audits

Table E.6: ResolutionCandidate Entity Schema

Field	Type	Constraint	Description
id	String	PK	Unique graph node identifier
normalizedRecordId	String	FK $\rightarrow$ NormalizedRecord	Corresponding canonical record
label	String	NOT NULL	Node classification label
firstSeenAt	DateTime	NOT NULL	Initial observation timestamp
lastSeenAt	DateTime	NOT NULL	Most recent observation timestamp

Table E.7: GraphNode Entity Schema

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
sourceNodeId	String	FK $\rightarrow$ GraphNode	Starting node for a directed edge
targetNodeId	String	FK $\rightarrow$ GraphNode	Terminating node for a directed edge
edgeType	String	NOT NULL	Relationship: linked, authored, replied, listed, reputation
weight	Float	NULLABLE	Edge weight or confidence value

Table E.8: GraphEdge Entity Schema

E.1.6 ResolutionCandidate**E.1.7 GraphNode****E.1.8 GraphEdge****E.1.9 Outbox**

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
eventType	String	NOT NULL	Category of the domain event
payload	JSON	NOT NULL	Serialized event data
createdAt	DateTime	DEFAULT now()	When the event was generated
publishedAt	DateTime	NULLABLE	When the event was dispatched successfully

Table E.9: Outbox Entity Schema

E.1.10 ProcessedMessage

Field	Type	Constraint	Description
id	Int	PK	Internal primary key
messageId	String	UNIQUE NOT NULL	Unique ID from the messaging queue
processedAt	DateTime	DEFAULT now()	Timestamp of successful idempotency check

Table E.10: ProcessedMessage Entity Schema

E.2 Entity-Relationship Summary

The core workflow is driven by the **Source** entity, which establishes the platforms to be monitored. A **CollectionRun** represents an invocation against a **Source** and yields multiple **Capture** entities. Each **Capture** acts as a pointer to raw payloads in MinIO.

Successfully parsed **Capture** instances map exactly one-to-one to a **NormalizedRecord** (often referred to as a Canonical Record). If parsing fails, the system bypasses this and creates a **Quarantine** entity referencing the failed **Capture**.

The **ResolutionCandidate** links two **NormalizedRecord** entities together for identity resolution, storing similarity metrics and human-in-the-loop audit data. Concurrently, normalized data populates the graph topology, where **GraphNode** entities wrap **NormalizedRecord** entities and are connected via **GraphEdge** relations representing semantic linkages (such as ‘authored’ or ‘replied’).

Finally, the **Outbox** and **ProcessedMessage** entities are infrastructural, managing transactional outbox patterns for domain event publishing and idempotency across asynchronous messaging queues.

Appendix F

Selected Source Code Excerpts

This appendix provides selected pseudocode excerpts to illustrate the foundational algorithms driving Wolverine’s ingestion, matching, and traversal logic.

F.1 Blocking Key Generation

This snippet demonstrates the generation of coarse blocking keys for partition-based filtering prior to extensive identity resolution.

```
1 function generateBlockingKeys(record: NormalizedRecord): string[] {
2   const handle = normalizeHandle(record.handle);
3   const prefix = handle.substring(0, 3);
4   const week = getRegistrationWeek(record.joinedAt);
5   const site = `${record.sourceId}.example.test`;
6   return [`${prefix}:${week}`, `${prefix}:${site}`];
7 }
```

Note: These are representative pseudocode excerpts derived from the source implementation, simplified for exposition.

F.2 Jaro-Winkler Similarity

This snippet outlines the core function signature and algorithmic weight application for measuring text proximity.

```
1 function jaroWinkler(s1: string, s2: string): number {
2   const jaro = jaroDistance(s1, s2);
3   const prefixLen = commonPrefixLength(s1, s2, 4);
4   return jaro + (prefixLen * 0.1 * (1 - jaro));
5 }
```

Note: These are representative pseudocode excerpts derived from the source implementation, simplified for exposition.

F.3 Composite Score Calculation

This snippet illustrates the weighted feature combination mechanism that generates an overall composite confidence score between two candidate records.

```
1 const score =
2   0.30 * jaroWinkler(a.handle, b.handle) +
3   0.15 * jaroWinkler(a.displayName, b.displayName) +
```

```

4   0.20 * temporalProximity(a.joinedAt, b.joinedAt) +
5   0.15 * 0.5 + // activityOverlap: hardcoded baseline
6   0.20 * crossSiteCue(a.handle, b.handle);

```

Note: These are representative pseudocode excerpts derived from the source implementation, simplified for exposition.

F.4 RAG Sanitization

This 4-rule pipeline demonstrates how raw scraped text is cleansed of control characters and command-injection vectors prior to processing.

```

1  function sanitizeText(text: string): string {
2    text = text.replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/g, '');
3    text = text.substring(0, 500);
4    text = text.replace(
5      /ignore previous|system:|<\\|\\[INST\\]/gi, '[REDACTED]'
6    );
7    return text;
8  }

```

Note: These are representative pseudocode excerpts derived from the source implementation, simplified for exposition.

F.5 BFS Traversal

This snippet details the bounded breadth-first search implementation used to rapidly explore node neighborhoods within the multi-tenant knowledge graph.

```

1  async function queryGraph(
2    startRef: string, maxDepth = 2, limit = 100
3  ): Promise<Subgraph> {
4    const visited = new Set<string>();
5    const queue: Array<[string, number]> = [[startRef, 0]];
6    while (queue.length > 0) {
7      const [nodeId, depth] = queue.shift()!;
8      if (depth > maxDepth || visited.size >= limit) break;
9      if (visited.has(nodeId)) continue;
10     visited.add(nodeId);
11     const neighbors = await getNeighbors(nodeId);
12     for (const n of neighbors) {
13       if (!visited.has(n.id)) queue.push([n.id, depth + 1]);
14     }
15   }
16   return buildSubgraph(visited);
17 }

```

Note: These are representative pseudocode excerpts derived from the source implementation, simplified for exposition.

Appendix G

Tor Gateway Security Controls

This appendix details the strict ingress routing, rate limiting, and request sanitization configured on the Tor-facing gateway used to safely expose specific read-only capabilities.

G.1 Gateway Architecture

The gateway relies on an Nginx Alpine container optimized for minimal surface area. It acts strictly with read-only proxy capabilities and is positioned within a dedicated DMZ network layer, isolating the internal architecture from direct external exposure.

G.2 HTTP Method Restrictions

To enforce immutability, only HTTP `GET` and `HEAD` methods are allowed through the gateway. Any other methods (e.g., `POST`, `PUT`, `DELETE`) immediately return a `405 Method Not Allowed` response. This globally prevents any database mutation or state changes via public ingress.

G.3 Rate Limiting

Traffic shaping policies are tightly enforced at the Nginx edge:

- **Rate:** Capped at 10 requests per second per client IP address.
- **Burst:** Allowed burst size of 20 to accommodate brief natural spikes without immediate punitive dropping.
- **Enforcement:** Excess requests automatically receive a `429 Too Many Requests` status code.

G.4 Administrative Endpoint Blocking

Several internal and administrative routes are explicitly blacklisted at the gateway level. Requests to these paths are intercepted and denied before reaching any backend service:

- `/metrics`: blocked (returns `403 Forbidden`)
- `/health/ready`: blocked
- `/admin`: blocked
- `/seed`: blocked

These endpoints exist strictly for internal instrumentation and automated provisioning.

G.5 Security Headers

Every outbound HTTP response dispatched by the proxy is instrumented with hardening headers:

- **Content-Security-Policy:** Enforces a highly restrictive policy minimizing client-side script execution.
- **X-Frame-Options: DENY:** Defends against clickjacking by preventing iframe embedding.
- **X-Content-Type-Options: nosniff:** Instructs browsers to strictly honor the declared MIME type, mitigating sniffing attacks.

G.6 RBAC Header Injection

To integrate with backend identity verification, the proxy performs role-based header injection:

- Public routes inject **X-Wolverine-Role:** `"researcher"`.
- Demonstration routes inject **X-Wolverine-Role:** `"public_demo"`.

Both injected roles carry exclusively read-only permissions on the backend API layer, guaranteeing that even if a payload traverses the gateway, it lacks the authorization to modify records.

G.7 Upstream Configuration

The proxy establishes upstream `proxy_pass` targets directly to the internal `wolverine-api` service, functioning as a reverse proxy that shields the backend from direct resolution.

G.8 Automated Security Testing

Gateway configuration integrity is validated via `scripts/test-tor-safety.sh`, which runs continuously as part of the integration pipeline. The script:

- Executes a comprehensive cURL suite validating 403 and 405 responses on restricted paths.
- Iteratively tests and verifies the rate limiting behavior and burst thresholds.
- Actively confirms that all administrative endpoint blocking is functioning as defined.

Bibliography

- [1] K. Soska and N. Christin, “Measuring the longitudinal evolution of the online anonymous marketplace ecosystem,” in *Proceedings of the 24th USENIX Security Symposium*, pp. 33–48, 2015.
- [2] X. Liao, K. Yuan, X. Wang, Z. Li, L. Xing, and R. Beyah, “Acing the IOC game: Toward automatic discovery and analysis of open-source cyber threat intelligence,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pp. 755–766, ACM, 2016.
- [3] I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.
- [4] M. A. Jaro, “Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida,” *Journal of the American Statistical Association*, vol. 84, no. 406, pp. 414–420, 1989.
- [5] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in *Proceedings of the Section on Survey Research Methods*, pp. 354–359, American Statistical Association, 1990.
- [6] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, “A comparison of string distance metrics for name-matching tasks,” in *Proceedings of the IJCAI-03 Workshop on Information Integration on the Web*, pp. 73–78, 2003.
- [7] G. Navarro, “A guided tour to approximate string matching,” *ACM Computing Surveys*, vol. 33, no. 1, pp. 31–88, 2001.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 3rd ed., 2009.
- [9] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024.
- [10] F. Perez and I. Ribeiro, “Ignore this title and hack this LLM: Investigating prompt injection attacks in large language models,” *arXiv preprint arXiv:2211.09527*, 2022.
- [11] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology — CRYPTO ’87*, pp. 369–378, Springer, 1987.
- [12] L. Lamport, R. Shostak, and M. Pease, “The Byzantine generals problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
- [13] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. Available at: <https://bitcoin.org/bitcoin.pdf>.

- [14] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Vertica, “Duplicate record detection: A survey,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 1, pp. 1–16, 2007.
- [15] L. Getoor and A. Machanavajjhala, “Entity resolution: Theory, practice and open challenges,” *Proceedings of the VLDB Endowment*, vol. 5, no. 12, pp. 2018–2019, 2012.
- [16] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer Science & Business Media, 2012.
- [17] A. Narayanan and V. Shmatikov, “Robust de-anonymization of large sparse datasets,” in *IEEE Symposium on Security and Privacy*, pp. 111–125, IEEE, 2008.
- [18] M. E. J. Newman, “The structure and function of complex networks,” *SIAM Review*, vol. 45, no. 2, pp. 167–256, 2003.
- [19] B. Nowok, G. M. Raab, and C. Dibben, “synthpop: Bespoke creation of synthetic data in R,” *Journal of Statistical Software*, vol. 74, no. 11, pp. 1–26, 2016.
- [20] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [21] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90, ACM, 2023.
- [22] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, “Prompt injection attack against LLM-integrated applications,” *arXiv preprint arXiv:2306.05499*, 2024.
- [23] M. Castro and B. Liskov, “Practical Byzantine fault tolerance and proactive recovery,” *ACM Transactions on Computer Systems (TOCS)*, vol. 20, no. 4, pp. 398–461, 2002.
- [24] B. Laurie, A. Langley, and E. Kasper, “RFC 6962: Certificate Transparency.” RFC Editor, 2013.
- [25] S. Josefsson and I. Liusvaara, “RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA).” RFC Editor, 2017.
- [26] S. Newman, *Building Microservices*. O’Reilly Media, 2nd ed., 2021.
- [27] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” *IEEE Access*, vol. 7, pp. 52976–52996, 2019.
- [28] A. Shostack, *Threat Modeling: Designing for Security*. John Wiley & Sons, 2014.
- [29] National Institute of Standards and Technology, “Guide for conducting risk assessments (NIST SP 800-30 Rev. 1).” NIST Special Publication, 2012.
- [30] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The second-generation onion router,” in *Proceedings of the 13th USENIX Security Symposium*, 2004.
- [31] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [32] C. Dwork, “Differential privacy,” in *Automata, Languages and Programming — ICALP 2006*, pp. 1–12, Springer, 2006.